

Planning with the Views

Kangrui Wang¹, Linjie Li², Zhengyuan Yang³, Shiqi Chen⁴, Zihan Wang¹, Li Fei-Fei⁵,
Jiajun Wu⁵, Leonidas Guibas⁵, Lijuan Wang³, Manling Li¹

¹Northwestern University ²University of Washington ³Microsoft
⁴University of Oxford ⁵Stanford University

Can VLMs predict how each camera move changes the view, and plan many such moves ahead? We call this ability *view planning*: using camera moves as planning primitives to find a target view in 3D. We study view planning in VIEWSUITE, built on ScanNet (Dai et al., 2017) with full 6-DoF camera pose control, and decompose it into two abilities: *tracking* how given camera actions change the view, and *composing* a path that localizes an unseen target view. Across 13 frontier VLMs, a sharp planning gap emerges: models track local view transitions but collapse when they must plan toward an unseen target view. This planning inability cannot simply be fixed by reinforcement learning (RL): with success near 2.5%, reward is too sparse for RL to bootstrap. Our key insight is to distill valid view transitions from on-policy self-exploration trajectories, by aggregating them into a *view graph* and distilling it into supervised demonstrations. Without any stronger teacher, this lifts Qwen2.5-VL-7B from 2.5% to 47.8% on interactive view planning, surpassing GPT-5.4 Pro (19.9%) and Gemini 3.1 Pro (21.3%). View planning offers a clean probe for prospective spatial reasoning: mentally looking ahead, predicting how future viewpoint changes will reshape observation, and inferring the camera pose of a target view before it is fully observed. Frontier VLMs still lack this capability, and the learned spatial priors transfer to other view-understanding tasks.

Website: <https://viewsuite.github.io>

Code: <https://github.com/ml-lab-nu/ViewSuite>

Models: <https://huggingface.co/collections/MLL-Lab/viewsuite-models>

Data: <https://huggingface.co/collections/MLL-Lab/viewsuite-datasets>

1. Introduction

To understand a 3D scene, an agent must be able to situate a view within it: infer where the camera stands, actively choose where to look next, and reason about how moving the camera can bring another view into sight. We call this **View Planning**, localizing a target view by composing a path of camera actions. It factors into two coupled abilities: *tracking* how a path changes the view, and actively *composing* a path to find where an unseen target view was taken. Intuitively, this is the difference between reading a given path and composing one from scratch. The first is local, reasoning relative to the current view. The second is global, requiring *prospective spatial reasoning*: anchoring the current and target views in a shared allocentric frame, mentally rolling out how camera actions update observations, and looking ahead through a sequence of view transitions to compose a multi-turn plan toward the target view.

Table 1 | Comparison with existing view reasoning benchmarks. VIEWSUITE takes the next step: multi-turn view planning in real-world 3D scenes with full 6-DoF camera pose control, at 165K instances. *EmbodiedBench supports multiple action types. Dashes: not applicable or not reported.

Benchmark	Task	Scale	Real World	3D	Action Space	Multi-turn
ViewSpatial-Bench (Li et al., 2025a)	View-Centric QA	5.7K	✓		–	
VSI-Bench (Yang et al., 2025a)	Video QA	5K	✓	✓	–	
CameraBench (Lin et al., 2025)	Video QA	3K	✓		–	
MindCube (Wang et al., 2025e)	View-Centric QA	21K		✓	–	
V* (Wu and Xie, 2024)	Visual Search	–	✓		2-DoF	✓
ActiView (Wang et al., 2025g)	Visual Search	3K	✓		2-DoF	✓
H*Bench (Yu et al., 2025)	Visual Search	–	✓	✓	2-DoF	✓
HM3D-OVON (Yokoyama et al., 2024)	Embodied Agent	–	✓	✓	4-DoF	✓
EmbodiedBench (Yang et al., 2025b)	Embodied Agent	1.1K		✓	6-DoF*	✓
Theory of Space (Zhang et al., 2026)	Embodied QA	2.7K		✓	2-DoF	✓
VIEWSUITE (Ours)	Visual Search	165K	✓	✓	6-DoF	✓

It is an instance of active perception (Bajcsy, 1988; Aloimonos et al., 1988; Bajcsy et al., 2018), where an agent actively chooses where to look and how to move the camera to change its view. Prior view search has advanced from 2D image regions (Wu and Xie, 2024; Wang et al., 2025g) to 360° panoramas (Yu et al., 2025). We take the next step: *multi-step view planning in real 3D scenes*, where the agent controls the full 6-DoF camera pose. Success is scored by how accurately the agent estimates the target view’s camera pose, not by whether it physically reaches an object or a physical destination. Thus, view planning targets a capability beyond navigation: mentally simulating an unobserved viewpoint and localizing it only by what it shows. The core challenge is not simply to move through views, but to plan with views: to mentally look ahead, forecast how actions change future observations, and infer where the target view lies before it is fully observed. The agent must use views as *planning primitives*.

Our setting differs from previous view search in three ways (Table 1): (1) *real 3D scenes* rather than synthetic graphics; (2) *6-DoF camera pose control*, where the camera moves with full six degrees of freedom, different from physical affordance, embodied navigation, and 2D image cropping; and (3) *multi-turn view composition* rather than single-step decisions.

We study whether current VLMs have this ability by building VIEWSUITE, a view-planning environment with full 6-DoF camera pose control on 286 real ScanNet (Dai et al., 2017) indoor scenes, yielding ~55K view pairs and ~165K task instances across three diagnostic tasks¹. As shown in Figure 1, in **Path-to-View** (P2V) and **View-to-Path** (V2P), the path is given, explicitly in the question or among the answer choices. In a *single turn*, VLMs need only *read* it: predict the view an action sequence produces, or infer which sequence connects two views. In **Interactive View Planning** (IVP), no path is given: VLMs must *compose* one, issue actions over *multiple turns* to localize an unseen target view, then submit a 6-DoF estimate of it. P2V and V2P thus ask whether VLMs can understand view transitions, while IVP asks whether they can use that understanding, an any-view-to-any-view problem that subsumes the tracking the first two require and adds the harder demand of placing a target view they have not yet seen.

Evaluating 13 frontier VLMs reveals a sharp planning gap: the best models can track local view changes over a short horizon (~70% on P2V/V2P) but collapse when they must compose them into a plan (at most 21.3% and most below 10% on IVP). The failure is driven neither by

¹The IVP success threshold is calibrated against human judgments via an alignment study (Appendix A.3).

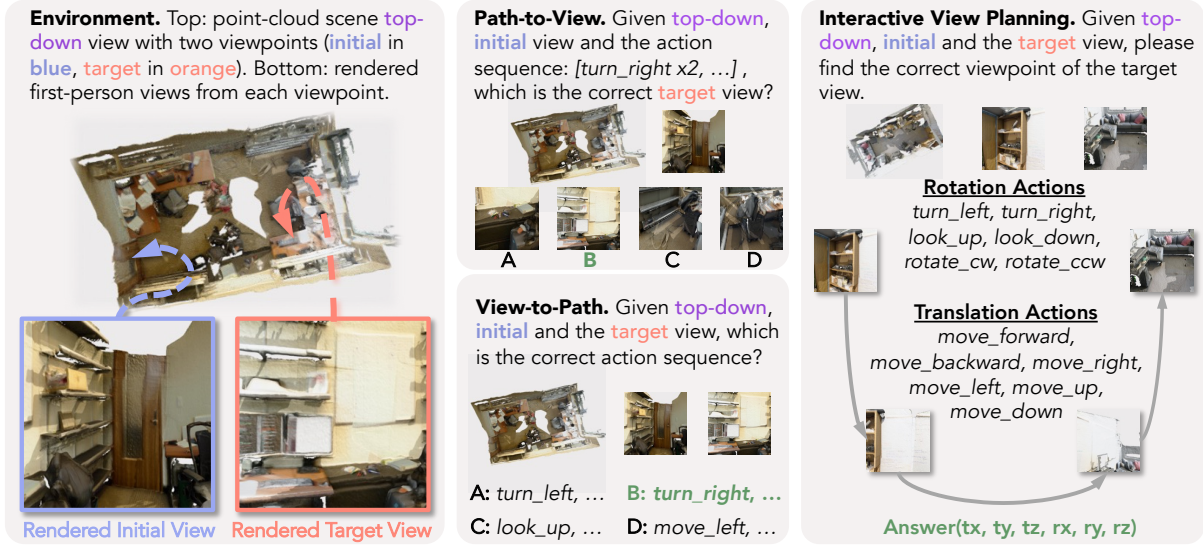


Figure 1 | Overview of VIEWSUITE. **Left:** Point cloud environment built on ScanNet with rendered first-person views. **Middle:** Path-to-View (P2V; predict the resulting view from an action sequence) and View-to-Path (V2P; infer the action sequence from two views), both single-turn. **Right:** Interactive View Planning (IVP), a multi-turn task where the agent plans view changes to localize the target view and submits a camera pose estimate.

exploration budget nor by rendering fidelity: more turns do not close the gap, and a higher-fidelity renderer leaves IVP essentially unchanged. What predicts failure is view distance: rotation distance for the tracking tasks, position distance for planning. The few successes reveal a sharper pattern: across models, over 90% of successful IVP runs occur only after the target view becomes observable. In other words, models usually solve IVP by moving until they happen to see the target view, then matching it, rather than by inferring where the target view lies beforehand. The planning gap is therefore more precisely a *cognitive gap*: even with the global top-down map in hand, frontier VLMs can rarely anchor egocentric views onto the map, mentally simulate how camera actions change those views, or localize a target view before seeing it. Prospective spatial reasoning is thus a harder, higher-level capability than tracking.

A natural way to learn planning is reinforcement learning (RL), yet this gap makes RL surprisingly ineffective. With a naive policy succeeding only $\sim 2.5\%$, reward is too sparse to reinforce: direct PPO plateaus at 3.2%, GRPO with reward-variance filtering reaches 5.2%, and bootstrapping from successful trajectories (SFT) reaches 6.2%. Our way past this bottleneck comes from a simple observation: every trajectory, successful or not, traces valid view transitions.

Distilling valid view transitions from raw exploration is then the central challenge. We construct a *view graph*, an any-view-to-any-view map assembled from the agent’s on-policy self-exploration. We then distill it into supervised demonstrations for view planning, and alternate this distillation with further self-exploration. As the policy improves, its exploration grows the view graph outward iteration by iteration, and the resulting supervision remains matched to the region of view space the agent can actually plan over. Like on-policy distillation (Lu and Thinking Machines Lab, 2025), our framework learns from the agent’s on-policy exploration; unlike it, there is no stronger teacher to imitate, and the teacher is the environment itself, whose structure the agent reveals by moving through it.

This iterative self-exploration and view-graph distillation lifts Qwen2.5-VL-7B from 2.5% to 47.8% on IVP, surpassing GPT-5.4 Pro (19.9%) and Gemini 3.1 Pro (21.3%). Building the

view graph from random rather than on-policy trajectories reaches only 13.0%, confirming that effective supervision must grow with the policy. Beyond these, we show that interactive view planning yields *transferable spatial priors*: under identical post-training, our model outperforms its base counterpart on other related view-understanding tasks. Together, view planning moves toward prospective spatial cognition (Tolman, 1948; Schacter et al., 2007): whether VLMs can internalize a world model of view transitions, plan over possible future views, and localize an unseen viewpoint.

2. VIEWSUITE: Problem Formulation, Environment, and Benchmark

VIEWSUITE casts view planning as a multi-turn decision process: an agent issues camera actions that alter its 6-DoF pose in a 3D point-cloud environment built on real ScanNet (Dai et al., 2017) indoor scenes, and at termination submits a camera-pose estimate of the target view, scored by distance to the ground truth (formal MDP and reward in Section 4, Eq. 3).

2.1. Problem Formulation

We decompose view planning into two coupled abilities: (1) *tracking* how a given path changes the view, and (2) *composing* a path that localizes an unseen target view. We design three complementary tasks targeting these abilities: P2V and V2P test tracking when the path is given, while IVP requires composing a plan when it is not.

Path-to-View (P2V). Given an initial view, a top-down reference, and an action sequence, the model predicts the resulting view from four options (multiple-choice), testing whether it can mentally simulate view transitions (Figure 7).

View-to-Path (V2P). Given initial and target views plus a top-down view, the model identifies which action sequence was executed, again from four options. P2V and V2P together test view-transition tracking in both directions (Figure 8).

Interactive View Planning (IVP). Given an initial view, a target view, and a top-down reference, the agent issues multiple actions per turn, observes the resulting view and camera pose, and within a fixed turn budget submits a 6-DoF camera pose estimate of where the target view was taken. Unlike the single-turn P2V and V2P, IVP requires the agent to plan a sequence of view changes over multiple turns to localize the target view (Figure 9).

2.2. Camera Pose Control Interface

VIEWSUITE exposes 6-DoF camera pose control through 12 step-size-parameterized actions: six translations (`move_forward`, `move_backward`, `move_left`, `move_right`, `move_up`, `move_down`) and six rotations (`turn_left`, `turn_right`, `look_up`, `look_down`, `rotate_cw`, `rotate_ccw`); full geometric definitions are in Appendix A.1 (Figure 1).

Step sizes are discretized so that a model controls the camera by *selecting* actions rather than specifying precise motion parameters, while each action still produces a visibly distinct view. The interface is decoupled from the 3D backend (point clouds, meshes, or simulators) and renders with Open3D (Zhou et al., 2018).

2.3. Data Collection and Evaluation

To construct the task data, we sample initial-target view pairs from ScanNet video frames. We use fixed step sizes $s_t=0.5$ m for translation and $s_r=30^\circ$ for rotation. We conduct scene-level and pair-level filtering, then reformat each pair into P2V, V2P, and IVP instances (detailed in Appendix A.2). This yields ~ 55 K view pairs across 286 ScanNet scenes.

Dataset splits. From the ~ 55 K view pairs, we split pairs within each scene at 1:10 ratio to form VIEWSUITE-5K and VIEWSUITE-50K; all experiments in this paper use VIEWSUITE-5K, with the full set reserved for future scaling studies. Scenes are then partitioned into train/dev/test at 8:1:1 ratio, ensuring no scene overlap across splits. Each view pair yields three task instances (P2V, V2P, and IVP), giving ~ 15 K instances in VIEWSUITE-5K, with the test set containing 530 pairs $\times$ 3 tasks = 1,590 instances. In total, VIEWSUITE provides ~ 165 K task instances.

Evaluation metrics. For P2V and V2P, we use accuracy (correct option selected). For IVP, we use *Success Rate*, defined by pose distance. We extract 6-DoF camera poses from camera-to-world extrinsic matrices, decomposing each into a position vector $\mathbf{t} \in \mathbb{R}^3$ and a rotation matrix $R \in \text{SO}(3)$. Translation distance and rotation distance between two poses are

$$d_{\text{pos}} = \|\mathbf{t}_1 - \mathbf{t}_2\|_2, \quad d_{\text{rot}} = \arccos\left(\frac{\text{tr}(R_1^T R_2) - 1}{2}\right). \quad (1)$$

The agent succeeds when $d_{\text{pos}} \leq \beta_t \cdot s_t$ and $d_{\text{rot}} \leq \beta_r \cdot s_r$ for threshold multipliers β_t, β_r . We calibrate β_t and β_r against human judgments of whether two rendered views depict the same place, selecting the combination that maximizes F_1 agreement with those judgments (full study in Appendix A.3). The best setting is $\beta_t=1$, $\beta_r=1$, i.e., one step size in each dimension (details in Table 9).

Unified view distance. We combine translation and rotation distance into a single difficulty score, the *unified view distance*:

$$d = \sqrt{(d_{\text{pos}}/s_t)^2 + (d_{\text{rot}}/s_r)^2}, \quad (2)$$

where normalizing by the step sizes makes each unit of d approximately correspond to one atomic action, so d approximates the length of the shortest action plan connecting two views. Across the 530 test pairs, d ranges from 1.4 to 6.8 with mean 3.7 (Figure 6 in Appendix A). We split test pairs into SHORT ($d < 3$; 185 pairs) and LONG ($d \geq 3$; 345 pairs) subsets, and further decompose along the translation and rotation axes for finer-grained diagnosis in Section 3.2.

3. Planning Gap in Frontier VLMs

3.1. Single-turn Tracking Understood, Multi-turn Planning Collapses

We evaluate 13 frontier VLMs (7 proprietary, 6 open-weight) plus a random-response baseline, detailed in Table 2. The central finding is a planning gap: frontier VLMs track local view transitions but cannot compose them into a multi-turn plan that localizes a target view. On P2V and V2P, the best models achieve a modest $\sim 50\%$ overall, rising to over 70% on short-horizon samples, well above the 25% MCQ chance baseline but far from solving the task. This indicates non-trivial but partial knowledge of view-action mappings, which degrades further on long-horizon samples that require mentally simulating cumulative transformations. When the model

Table 2 | **The planning gap in frontier VLMs.** Main evaluation results on VIEWSUITE: accuracy (%) for P2V/V2P and success rate (%) for IVP², on Short (view distance $d < 3$) / Long ($d \geq 3$) / Overall splits. The best models exceed 70% on short-horizon P2V/V2P but reach at most 21.3% on IVP, with most below 10%. Proprietary: GPT (OpenAI, 2026a,b), Gemini (Google DeepMind, 2025, 2026), Claude (Anthropic, 2026), Grok (xAI, 2026). Open-weight: Qwen (Bai et al., 2025a,b; Qwen Team, 2026), GLM (Hong et al., 2025), Kimi (Kimi Team, 2026). Models sorted by Overall within each group; best per column in **bold**.

Model	Path-to-View (P2V)			View-to-Path (V2P)			Interactive View Planning (IVP)			Overall
	Short	Long	All	Short	Long	All	Short	Long	All	
Random Response	20.7	24.6	23.3	24.3	26.5	25.7	2.2	0.0	0.8	16.6
<i>Proprietary Models</i>										
GPT-5.4 Pro	70.8	43.8	53.2	72.4	38.8	50.6	34.8	11.7	19.9	41.2
Gemini 3.1 Pro	63.8	40.9	48.9	53.0	47.5	49.4	28.6	17.4	21.3	39.9
GPT-5.4	57.3	42.9	47.9	60.5	37.4	45.5	33.5	7.5	16.6	36.7
Grok 4.20 Beta	61.6	38.0	46.2	44.9	44.3	44.5	17.3	2.9	7.9	32.9
GPT-5.1	60.5	35.1	44.0	52.4	33.3	40.0	11.9	3.2	6.2	30.1
Claude Opus 4.6	46.5	28.4	34.7	47.6	38.3	41.5	23.8	3.8	10.8	29.0
Gemini 3 Pro	50.3	31.0	37.7	44.9	35.4	38.7	13.5	7.0	9.2	28.5
<i>Open-Weight Models</i>										
Qwen3.5-397B	57.8	30.1	39.8	44.3	31.0	35.7	12.4	0.0	4.3	26.6
GLM-4.6V	36.4	23.2	27.8	31.4	29.7	30.2	9.2	1.2	4.0	20.7
Qwen2.5-VL-72B	28.1	29.3	28.9	35.7	30.1	32.1	2.2	0.6	1.1	20.7
Qwen3-VL-32B	27.0	27.5	27.4	41.1	28.7	33.0	4.3	0.0	1.5	20.6
Kimi K2.5	36.2	24.6	28.7	18.4	29.3	25.5	4.9	1.2	2.5	18.9
Qwen2.5-VL-7B	23.8	32.5	29.4	27.0	22.9	24.3	7.0	0.0	2.5	18.7

must instead compose the path itself, performance collapses. On IVP the best model (Gemini 3.1 Pro) reaches only 21.3%, most models fall below 10%, and on long-horizon samples most fall below 3%; the gap holds across proprietary and open-weight models, with every open-weight model below 5%. Notably, GPT-5.4 Pro outperforms GPT-5.4 across all tasks and splits, including IVP; given that GPT-5.4 Pro is widely believed to be a test-time scaled variant of GPT-5.4, this suggests that additional test-time computation can meaningfully improve spatial reasoning on VIEWSUITE.

3.2. What Bottlenecks Interactive View Planning?

Does turn budget affect IVP performance? A natural hypothesis is that models fail at IVP simply because 10 turns is insufficient. We test this by increasing the turn budget to 20 and 30 for four proprietary models (Table 3). All models improve from 10 to 20 turns, with Claude Opus 4.6 showing the largest gain (nearly doubling). However, gains from 20 to 30 turns are marginal or zero for most models. This diminishing return suggests that IVP performance is bottlenecked by planning ability rather than exploration horizon, as models exhaust their effective strategies well before the turn limit.

²GPT-5.4 Pro refuses 23 of the 530 IVP test instances under its violation policy. Its IVP rates are reported over the remaining 507 valid instances (101/507=19.9%). All other models are evaluated on the full 530.

Table 3 | **Neither more turns nor sharper rendering closes the planning gap.** Left three blocks: interactive view planning (IVP) success rate (%) at turn budgets of $B=10, 20$, and 30 with point-cloud rendering; gains from 20 to 30 turns are marginal. Rightmost block: P2V / V2P / IVP All-split results at a turn budget of 10 with higher-fidelity Gaussian Splatting (GS) rendering; IVP improves only marginally.

Model	IVP, B = 10			IVP, B = 20			IVP, B = 30			GS, B = 10			
	Short	Long	All	Short	Long	All	Short	Long	All	P2V	V2P	IVP	Overall
Gemini 3.1 Pro	28.6	17.4	21.3	30.8	18.8	23.0	33.0	17.9	23.2	55.3	49.4	23.2	42.6
GPT-5.4	33.5	7.5	16.6	33.0	11.9	19.2	35.1	12.1	20.2	43.8	31.1	18.5	31.1
Grok 4.20 Beta	17.3	2.9	7.9	23.8	5.2	11.7	27.0	3.5	11.7	28.3	31.5	8.1	22.6
Claude Opus 4.6	23.8	3.8	10.8	31.9	10.1	17.7	35.7	11.0	19.6	35.3	41.3	12.3	29.6

Does rendering quality affect model performance? A natural concern is that point-cloud rendering, with its sparse and noisy pixels, may itself bottleneck the agent. We re-render the test set with 3D Gaussian Splatting (Kerbl et al., 2023), a higher-fidelity neural renderer, using pretrained per-scene 3DGS reconstructions of the ScanNet scenes from SceneSplat-7K (Li et al., 2025b), and re-evaluate four proprietary models on all three tasks at a turn budget of 10 (Table 3, rightmost block). The pattern across tasks is asymmetric: IVP improves consistently but only marginally ($+0.2$ to $+1.9$ points), whereas P2V and V2P show mixed and sometimes large changes: Gemini 3.1 Pro gains $+6.4$ on P2V, while GPT-5.4 and Grok 4.20 Beta lose 14.4 and 13.0 on V2P (relative to their point-cloud rendering scores in Table 2). That a higher-fidelity renderer does not unlock single-turn performance, and yields only modest gains on IVP, indicates that the IVP bottleneck is not the visual fidelity of the rendered observation but the model’s ability to compose view changes into a multi-turn plan. Combined with the test-time-compute gain above (GPT-5.4 Pro over GPT-5.4), this places the bottleneck in reasoning rather than perception: sharper pixels do not help, more thinking does.

Is rotation or translation the primary difficulty driver? Decomposing unified view distance into rotation and position axes (Figure 2) reveals contrasting difficulty drivers across task types. P2V/V2P degrade primarily with rotation distance (e.g., GPT-5.4 Pro loses ~ 25 points across rotation bins on P2V), since cumulative rotations are hard to mentally simulate. IVP reverses this: success collapses with position distance ($\sim 7\times$ drop for GPT-5.4 Pro), as 3D translation requires spatial layout understanding and path planning beyond simple orientation control. This contrast aligns with our tracking/planning decomposition: reading a path is largely a matter of *tracking*, which degrades with rotation distance, whereas composing a plan additionally requires reasoning about where an unseen target view lies in the scene layout, a demand that grows with translation rather than orientation.

Sample-level factor analysis (Spearman ρ across 12 geometric, visual overlap, and directional factors defined in Appendix B.3; full results in Appendix B.2) further confirms the position bottleneck for IVP and the rotation bottleneck for P2V/V2P.

Do successes reflect mental localization or view matching? Theoretically, a competent spatial-reasoning model need not see the target view to localize it: after a few informative moves, it could establish an internal spatial mental model, infer how the target view relates to the views it has observed, and submit the target view’s camera pose without visiting it. We test whether current successes of frontier VLMs work this way. For every successful IVP rollout, we check whether any *observed* camera pose (the initial view plus every view the agent moves

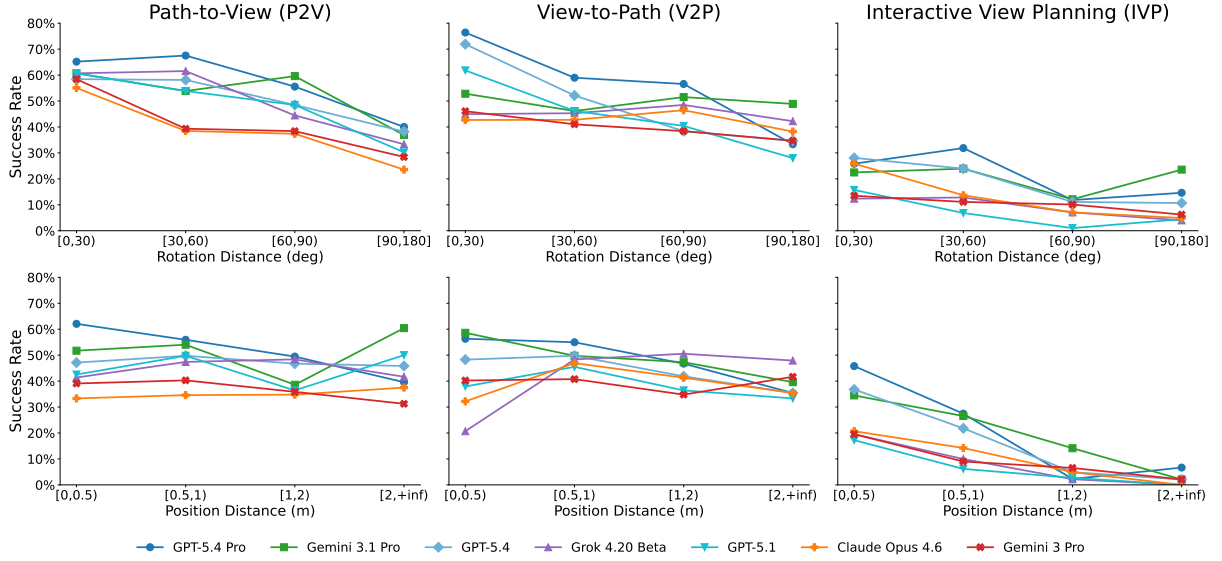


Figure 2 | **Tracking degrades with rotation distance, planning with position distance.** Success rate vs. rotation distance (top) and position distance (bottom) for proprietary models across all three tasks: P2V/V2P degrade primarily with rotation distance, while IVP success collapses with position distance.

Table 4 | **Successes reflect view matching, not mental localization.** Each model’s successful interactive view planning (IVP) rollouts, split by whether the agent ever observed a view within the success threshold (0.5 m / 30°) of the target view before answering: $\geq 90\%$ of successes follow such a visual encounter.

Model	#Success	Successful rollouts	
		Visited target view	Inferred (no visit)
GPT-5.4 Pro	101	99 (98.0%)	2 (2.0%)
Gemini 3.1 Pro	113	112 (99.1%)	1 (0.9%)
GPT-5.4	88	83 (94.3%)	5 (5.7%)
Grok 4.20 Beta	42	38 (90.5%)	4 (9.5%)
Claude Opus 4.6	57	54 (94.7%)	3 (5.3%)

to) falls within the success threshold (0.5 m / 30°) of the target view, *i.e.* whether the agent visually encountered the target view before answering (Table 4). Across all five models, $\geq 90\%$ of successes (up to 99.1% for Gemini 3.1 Pro) are coupled to such a visual encounter; genuine inference of a correct camera pose without ever visiting a threshold-close view accounts for at most $\sim 10\%$ of successes. This suggests that current VLMs mostly succeed by *view matching*, rather than by planning ahead through view space. The gap exposed by IVP is therefore not only a performance gap, but a cognitive gap: today’s models can move through views, but rarely use observed views to forecast and localize a target view that has not yet been fully seen. In short, what they lack is not movement, but planning.

4. Self-Exploration with View Graph Distillation

Section 3 shows that frontier VLMs track local view transitions but cannot compose them to localize an unseen target view. Tracking how a single action changes the view is fundamentally

different from planning: composing a sequence of actions whose accumulated observations pin down where an unseen target view lies. We ask whether an agent can bridge this gap through *self-exploration* alone: interacting with the environment, learning from its own experience, and improving without any external demonstration.

Although the action set superficially resembles embodied-navigation primitives, an IVP rollout is at heart a localization problem: actions move only the viewpoint and form a planned trajectory of view manipulations, while reward is granted for an accurate 6-DoF estimate of the target view, not for physically arriving at it. Success thus turns entirely on reasoning: actions serve only to gather evidence for a localization decision made within the turn budget. This raises a key question: with no external demonstrations and a naive policy succeeding only $\sim 2.5\%$ of the time, can the agent extract valid supervision from experience that is overwhelmingly unsuccessful?

4.1. Interactive View Planning as a Localization Problem

We model IVP as a finite-horizon decision process. At each turn t , the agent observes the rendered view o_t and its current 6-DoF camera pose $p_t \in \text{SE}(3)$, and selects an action $a_t \in \mathcal{A}$ from the 12-element action set. The environment updates the pose deterministically, $p_{t+1} = T(p_t, a_t)$, and renders the next view. After at most T turns the agent submits a target estimate $\hat{p}^* \in \text{SE}(3)$, scored by

$$r(\hat{p}^*, p^*) = \mathbf{1}[d_{\text{pos}}(\hat{p}^*, p^*) \leq \beta_t s_t \wedge d_{\text{rot}}(\hat{p}^*, p^*) \leq \beta_r s_r] + 0.1 \mathbf{1}_{\text{format}}, \quad (3)$$

where p^* is the ground-truth camera pose of the target view and $\beta_t = \beta_r = 1$ are the human-calibrated thresholds of Section 2.3. A learned policy π_θ maps the rollout history to the next action and, upon termination, to the target estimate.

4.2. Bootstrapping View Planning from Self-Exploration

The key observation is that every trajectory, whether or not it reaches its goal, traces *valid* view transitions through the scene: moving from one viewpoint to another is meaningful supervision regardless of the original target view. Aggregated across exploration, these transitions form a *view graph*, a compact map of how viewpoints connect across a scene, in which connectivity discovered by one episode becomes reusable for any other. This is exactly the any-view-to-any-view structure that composing a plan requires, and it is assembled entirely from the agent’s own moves, much as a person builds a spatial map in which even a wrong turn teaches which rooms connect to which hallways. Crucially, there is no stronger teacher here: the agent uncovers the environment’s structure by exploring it. In effect, the view graph is an empirical world model of view transitions, holding exactly what the model has experienced; distillation turns it into the model’s own.

We turn this into iterative training that alternates two stages (Figure 3). In each iteration, the agent explores the environment, incrementally compressing its trajectories into the view graph, and a distillation stage samples paths from the graph and reformulates them into supervised view-planning demonstrations that fine-tune the policy. The fine-tuned model initializes the next iteration, so exploration starts from familiar views and pushes outward round by round, growing the graph on-policy toward the views the model will later plan through.

Self-exploration stage. The agent interacts with the VIEWSUITE environment using PPO (Schulman et al., 2017) under a sparse reward: an outcome reward of +1 when the predicted camera

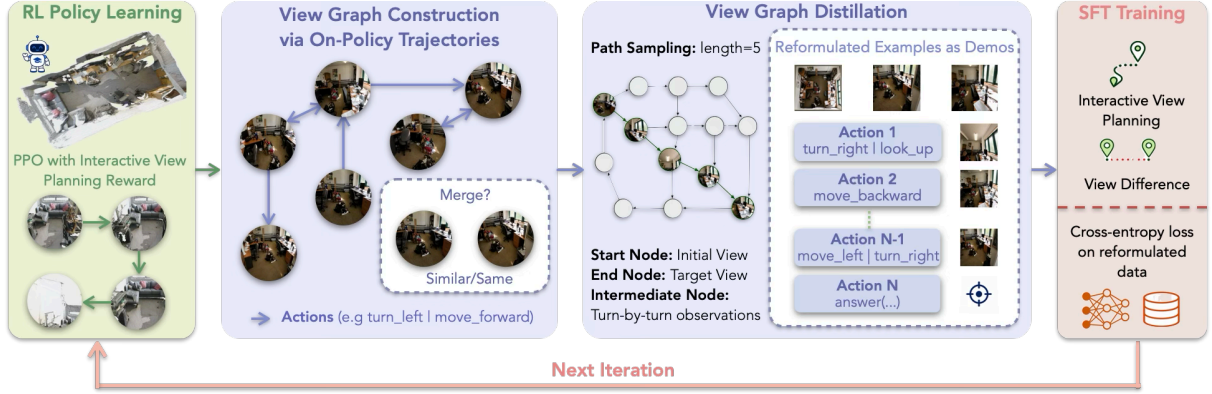


Figure 3 | Iterative training pipeline. **Left:** in the *self-exploration* stage, the agent actively explores VIEWSUITE environments under sparse outcome rewards; completed trajectories are continuously compressed into a view graph whose nodes are viewpoints and whose edges are actions. **Right:** in the *view graph distillation* stage, paths sampled from the graph are reformulated into multi-turn view-planning demonstrations and auxiliary supervision (view-difference estimation and its multiple-choice variant). The distilled model initializes the next self-exploration iteration, progressively bootstrapping the policy.

pose of the target view falls within the IVP success threshold ($d_{\text{pos}} \leq 0.5 \text{ m}$, $d_{\text{rot}} \leq 30^\circ$; Section 2.3) and 0 otherwise, plus a format reward of +0.1 for a correctly structured response. A background process continuously converts completed trajectories into the view graph: each node stores a viewpoint with its rendered view, and each directed edge stores the actions taken between two viewpoints. Nodes and edges are deduplicated by viewpoint similarity (Appendix C.4), so the graph does not grow unboundedly as regions are revisited.

View graph distillation via task reformulation. In the distillation stage, we sample paths from the accumulated graph and convert them into supervised data. The mechanism is *task reformulation*. For any path $P = (v_0, a_1, v_1, \dots, a_K, v_K)$ in the graph, define the operator

$$\mathcal{R}(P) = (o_{\text{init}} = v_0, o_{\text{target}} = v_K, (a_1, \dots, a_K), \hat{p}^* = p_{v_K}), \quad (4)$$

which yields a valid IVP demonstration regardless of whether the original episode succeeded: the end node becomes the target, the start node becomes the initial view, and the action chain becomes the target action sequence. Because P may begin and end at arbitrary nodes, every path is an any-view-to-any-view demonstration, the lever that lets us draw dense supervision from raw, mostly-failed exploration (Algorithm 3 in Appendix C.1). From the same graph we generate three supervision types: (1) multi-turn view planning via reformulation (the primary task), (2) view-difference estimation (predicting the unified view distance between two views), and (3) a multiple-choice variant of view-difference estimation (Appendix C.5). The policy is trained with a standard cross-entropy loss using LLaMA-Factory (Zheng et al., 2024); the self-exploration stage is built on VAGEN (Wang et al., 2025c) and veRL (Sheng et al., 2024).

5. Self-Exploration Closes the Gap

5.1. Experimental Setup

We instantiate our framework on two base models: Qwen2.5-VL-7B-Instruct (Bai et al., 2025a) as the primary base, and Qwen3-VL-8B-Instruct as a robustness check. In both cases, iterative

training runs for four iterations on 3,419 VIEWSUITE training environments with up to 10 turns per episode (training/validation split in Appendix C.6). The first three iterations alternate a 60-step self-exploration stage with 3 epochs of view graph distillation for rapid bootstrapping, and the final iteration runs self-exploration to convergence without further distillation. Full hyperparameters are provided in Appendix C.2 (RL) and Appendix C.3 (SFT).

Prompting Baselines. We include the untrained Qwen2.5-VL-7B-Instruct, GPT-5.4 Pro (OpenAI, 2026b), and Gemini 3.1 Pro (Google DeepMind, 2026) as zero-shot reference points.

Training Baselines. We compare against three RL methods, all trained from Qwen2.5-VL-7B-Instruct on the same environments with identical reward and action space:

- **Direct PPO.** PPO (Schulman et al., 2017) training from the base model without any distillation stage. This tests whether self-exploration alone can succeed given sufficient training steps.
- **Direct GRPO (filter).** GRPO (Shao et al., 2024) with $n=4$ rollouts per prompt and reward-variance-based filtering (Wang et al., 2025f). This tests whether an alternative RL algorithm with implicit best-of- n selection can bootstrap learning.
- **Success-Only Bootstrapping.** Iterates between PPO and SFT like our framework, but constructs SFT data by filtering successful RL trajectories (reward > 0.5) rather than sampling from a view graph with task reformulation. This isolates the contribution of our framework’s graph-based data generation from any trajectory, including failures.

Training Ablations. We evaluate three ablations of our framework on Qwen2.5-VL-7B-Instruct:

- **1 iter + RL and 2 iter + RL.** Stop after fewer iterations to measure the contribution of the view graph distillation stage.
- **Random-graph.** Builds the view graph from a random action generator instead of model-collected trajectories, isolating the contribution of on-policy graph construction.

5.2. Closing the Gap on Interactive View Planning

As shown in Table 5, our framework improves Qwen2.5-VL-7B-Instruct from 2.5% to 47.8% on IVP, surpassing all frontier models. Applied to Qwen3-VL-8B-Instruct, the same framework reaches 32.5%, above every prompting and training baseline and above the strongest frontier model (Gemini 3.1 Pro, 21.3%). The gains thus hold across pre-trained backbones, though their absolute magnitude is backbone-dependent. All three training baselines remain below 7%: Direct PPO (3.2%) confirms that self-exploration alone cannot succeed when the base success rate is near zero; Direct GRPO with filtering (5.2%) shows that best-of- n selection helps only marginally; and Success-Only Bootstrapping (6.2%) underperforms our framework, highlighting the importance of view-graph construction and task reformulation that generate useful supervision from *any* trajectory rather than only successful ones. Iteration ablations show progressive improvement (12.0% $\rightarrow$ 27.9% $\rightarrow$ 47.8%) across 1, 2, and 3 iterations, while the Random-graph variant achieves only 13.0%, confirming that on-policy graph construction is critical, echoing the on-policy advantage of DAgger (Ross et al., 2011) and on-policy distillation (Agarwal et al., 2024). Graphs built from random-action trajectories cover regions of view space the model rarely visits during evaluation, so the resulting reformulated supervision transfers poorly. As the policy improves each round, it explores farther and the graph grows outward, covering more of the space the model must plan over. The ranking between methods is preserved under two evaluation-protocol relaxations, No-Snap (raw rotations executed as-is) and No-Submit (success

Table 5 | **Self-exploration with view graph distillation mitigates the planning gap.** Interactive view planning (IVP) success rates (%) on the VIEWSUITE test set (Short: view distance $d < 3$; Long: $d \geq 3$): our framework lifts Qwen2.5-VL-7B from 2.5% to 47.8%, above the best frontier model (21.3%), while all training baselines stay below 7% and the Random-graph ablation reaches only 13.0%.

Method	Short	Long	All
<i>Prompting Baselines</i>			
Qwen2.5-VL-7B-Instruct	7.0	0.0	2.5
GPT-5.4 Pro	34.8	11.7	19.9
Gemini 3.1 Pro	28.6	17.4	21.3
<i>Training Baselines</i>			
Direct PPO	7.0	1.2	3.2
Direct GRPO (filter)	10.8	2.2	5.2
Success-Only Bootstrapping	14.0	2.0	6.2
<i>Training Ablations</i>			
Random-graph	25.4	6.4	13.0
1 iter + RL	24.3	5.4	12.0
2 iter + RL	49.7	16.2	27.9
<i>Our Methods</i>			
Qwen2.5-VL-7B-Instruct	67.2	36.9	47.8
Qwen3-VL-8B-Instruct	56.8	19.4	32.5

the moment the pose enters the threshold), so the gain is not an artifact of rotation snapping or of the explicit submit step (Table 10, Appendix B.1).

5.3. What Has the Model Learned?

What exploration strategy does the trained model learn? We measure 3D point cloud coverage across turns: **target intersection ratio** (fraction of target view vertices covered) and **scene coverage ratio** (fraction of all scene vertices observed); see Appendix D.1 for details. Our trained model learns an effective exploration policy (Figure 4): scene coverage grows rapidly in early turns as the agent explores broadly, then plateaus; target intersection ratio accelerates in the middle turns as the agent moves toward the target view, reaching $\sim 55\%$. This two-phase pattern (explore then approach) is absent in the base model and frontier models, which show flat or erratic target coverage throughout (full model comparison in Appendix D.1). However, this behavior also suggests a limitation: our trained model still localizes largely by approaching until the target view becomes observable, rather than through prospective spatial reasoning that would localize it beforehand.

How does training reshape the model’s attention? We also analyze how training changes the model’s attention mechanism (Figure 5; methodology in Appendix D.2). The trained model allocates more attention to image tokens than the base model across most layers and turns. This suggests that view planning training changes how the model uses observations: instead of treating each view as a weak contextual cue, the model increasingly grounds its decisions in visual evidence accumulated over interaction. The rise of image attention in early and middle turns is consistent with active evidence gathering, while the drop near the final turn suggests a

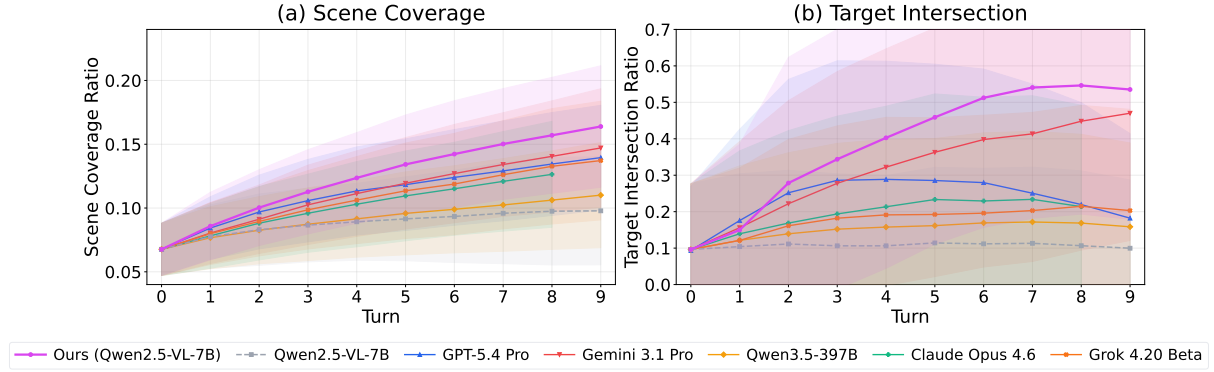


Figure 4 | **The trained model learns a two-phase explore-then-approach strategy.** Point-cloud coverage across interactive view planning (IVP) turns, averaged over the test set. **(a)** *Scene coverage ratio* (fraction of all scene vertices observed) rises steadily as the agent surveys the scene broadly. **(b)** *Target intersection ratio* (fraction of target view vertices covered) grows slowly at first, then accelerates through the middle turns as the agent approaches the target view, an explore-then-approach pattern unique to our trained model; the base and frontier models keep target coverage flat or erratic throughout (full comparison in Appendix D.1).

Table 6 | **View-planning priors transfer.** Under identical GRPO post-training, our model outperforms the base by 8–12 points on P2V/V2P (transfer within view understanding) and ~10 points on MindCube (transfer to an external benchmark), despite similar or lower starting points. Init/Post: accuracy before/after post-training; Base: Qwen2.5-VL-7B-Instruct; Ours: our IVP-trained model.

Model	P2V		V2P		MindCube	
	Init	Post	Init	Post	Init	Post
Base	32.1	45.1	29.2	44.8	33.0	56.3
Ours	25.7	57.3	31.6	52.8	33.1	66.2

transition from visual grounding to decision formation.

Do the learned priors transfer to other view-related tasks? We ask whether the spatial priors acquired through interactive view planning transfer to other view-related tasks under further fine-tuning. We test this under identical GRPO post-training on (i) P2V and V2P from VIEWSUITE, which share scenes and action space with IVP but require different reasoning, and (ii) MindCube (Wang et al., 2025e), an external benchmark with no shared scenes, actions, or rendering pipeline. Details are provided in Appendix D.3. Despite a slightly lower starting point, our trained model outperforms the base by 8–12 points on P2V and V2P after post-training (Table 6), indicating that view-planning experience yields priors that go beyond the IVP task itself. On the external MindCube benchmark, our model gains ~10 points over the base, showing that the priors transfer to view-dependent spatial reasoning even outside our environment. Interactive view planning is therefore not a narrow skill: it yields spatial priors that strengthen view understanding both within and beyond VIEWSUITE.

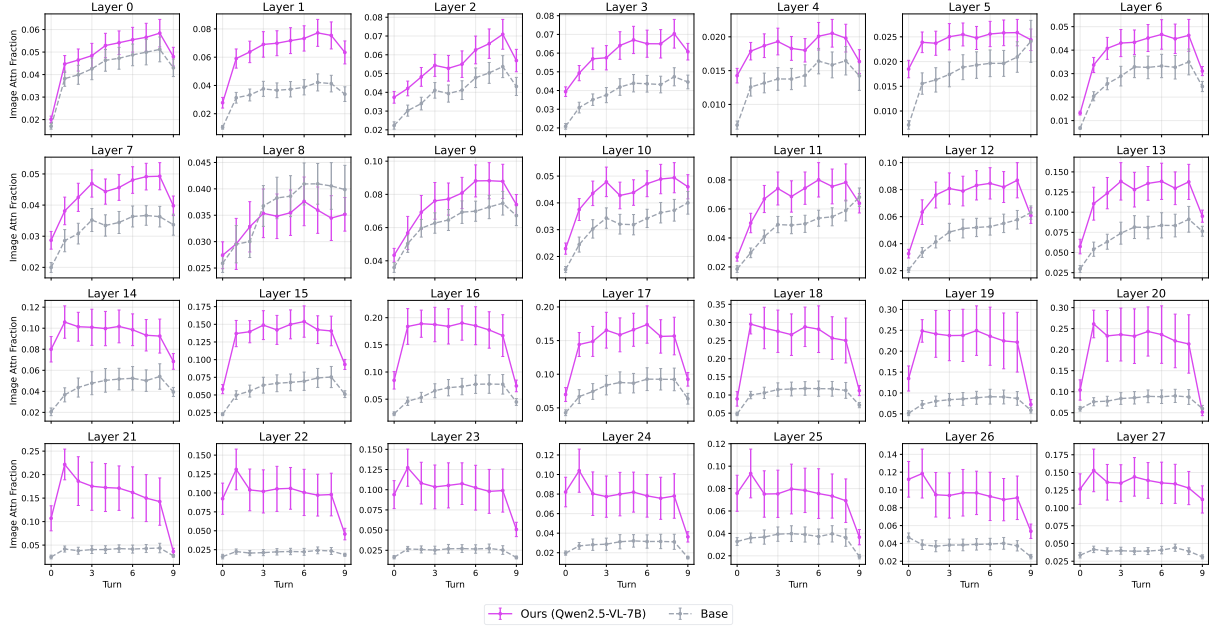


Figure 5 | **Training shifts attention toward visual evidence.** Image attention fraction (share of response-token attention on image tokens) across all 28 layers of trained and base Qwen2.5-VL-7B: the trained model attends to image tokens more across most layers and turns, with the largest gap in the mid-to-late layers; image attention rises in the early and middle turns and falls at the final turn.

6. Related Work

View reasoning and 3D scene benchmarks. View-centric QA benchmarks such as Mind-Cube (Wang et al., 2025e) and ViewSpatial-Bench (Li et al., 2025a) probe view-dependent reasoning from images but are non-interactive, as are broader static spatial-QA benchmarks (Cheng et al., 2024; Chen et al., 2024; Ma et al., 2025) and 3D scene QA built on real scans (Azuma et al., 2022; Ma et al., 2023; Hong et al., 2023). Multi-image and video benchmarks (Yang et al., 2025a; Lin et al., 2025; Zhang et al., 2025b; Yang et al., 2026b; Yeh et al., 2025) add cross-frame and temporal reasoning, and recent work pushes spatial supersensing (Yang et al., 2025c) and pose-grounded video understanding (Yang et al., 2026a) further, yet all treat the model as a passive observer of given frames. Embodied QA and embodied-agent benchmarks (Yang et al., 2025b; Yokoyama et al., 2024; Majumdar et al., 2024) introduce active interaction, but optimize for physical arrival at a semantic goal such as an object or a room, where success turns on affordance and traversability. The closest benchmark to ours is E3VS-Bench (Sakamoto et al., 2026), which studies viewpoint-dependent active perception in Gaussian-splatting scenes; VIEWSUITE differs in providing full 6-DoF control, a multi-turn planning task, and a training framework. More broadly, VIEWSUITE targets spatial *localization* through active view planning: the agent plans view manipulations to gather visual evidence, then submits a 6-DoF estimate of where the target view was taken. Success requires localization accuracy, not physical arrival, isolating viewpoint reasoning from embodied navigation. This framing echoes spatial cognition: humans build cognitive maps of their environment (Tolman, 1948; O’Keefe and Nadel, 1978), combine egocentric views with allocentric representations (Burgess, 2006), and mentally rotate what they see to anticipate unseen appearances (Shepard and Metzler, 1971), reasoning ahead in the manner of prospective cognition (Schacter et al., 2007). Recent benchmarks begin to ask the same questions of models, probing whether spatial cognition emerges in frontier models (Ramakr-

ishnan et al., 2025), whether LLMs build cognitive maps that support planning (Momennejad et al., 2023), and whether VLMs hold a coherent theory of space (Zhang et al., 2026), form spatial mental models from limited views (Wang et al., 2025e), or have internal world models of embodied interaction (Gao et al., 2025; Wang et al., 2025d; Li et al., 2024); view planning operationalizes this prospective spatial cognition as a measurable task.

Visual search and active perception. The closest prior work probes view planning through visual search, an instance of active perception (Bajcsy, 1988; Aloimonos et al., 1988; Bajcsy et al., 2018). ActiView (Wang et al., 2025g) restricts the action space to zoom and shift within a 2D image; V* (Wu and Xie, 2024) performs LLM-guided search inside a single high-resolution image; and H*Bench (Yu et al., 2025) studies head rotation over a 360° panorama. A recent line of agentic visual search trains VLMs to crop, zoom, and reason in pixel space (Lai et al., 2025; Zheng et al., 2025b; Wang et al., 2025a; Zhang et al., 2025a; Man et al., 2025), while embodied agents learn to explore 3D scenes and decide when to stop (Ren et al., 2024; Yang et al., 2024; Chaplot et al., 2020). Because our success metric is a 6-DoF pose estimate, our task also connects to learning-based camera-pose and geometry estimation (Wang et al., 2024, 2025b; Schönberger and Frahm, 2016), where VLMs are known to struggle (Deng et al., 2026). VIEWSUITE extends the visual-search thread to real 3D scenes with full 6-DoF camera pose control, where the agent must compose a multi-turn plan to localize a target view rather than crop or rotate within a fixed vantage (Table 1).

Agentic RL and learning from failure. Outcome-supervised RL substantially improves LLM reasoning (Shao et al., 2024; DeepSeek-AI, 2025), and follow-up work extends it to agentic and multi-modal settings (Zheng et al., 2025a; Sheng et al., 2024; Wang et al., 2025f,c). A parallel line of self-improvement bootstraps a policy from its own generations by fine-tuning on successful trajectories (Zelikman et al., 2022; Singh et al., 2024; Gulcehre et al., 2023; Hosseini et al., 2024; Qu et al., 2024; Oh et al., 2018; Qin et al., 2025; Ma et al., 2026). Distilling supervision from a model’s own on-policy rollouts, rather than from a fixed off-policy dataset, is also central to on-policy distillation and online imitation learning (Ross et al., 2011; Rusu et al., 2016; Agarwal et al., 2024; Lu and Thinking Machines Lab, 2025); our view graph is assembled entirely from such on-policy exploration, and we confirm that on-policy construction is critical (Section 5). In classical RL, Hindsight Experience Replay (Andrychowicz et al., 2017; Zhang et al., 2023) relabels failed trajectories with the goals they happened to achieve, and recent work adapts this to LM agents by rewriting failed rollouts into supervised targets (Hu et al., 2025). Our framework builds on this idea, since relabeling a path’s endpoint as its target is one instance of our reformulation operator $\mathcal{R}$ (Eq. 4), but its aim is broader. Rather than densifying reward episode by episode, we aggregate *all* rollouts into a view graph that captures how viewpoints connect across a scene, and distill this any-view-to-any-view structure into diverse supervised reformulations: multi-turn view planning, view-difference estimation, and its multiple-choice variant. Distilling these reformulations reshapes the policy distribution so that subsequent RL rollouts reach high-reward trajectories more often, combining the distribution sharpening of RL with the distribution reshaping of SFT to overcome the sparse reward under which pure RL fails.

7. Conclusion and Limitations

We study *view planning*: composing camera actions into multi-turn plans that localize an unseen target view. Across frontier VLMs, we reveal a clear planning gap: models can track local view transitions, but struggle to compose them into plans toward a target view before it becomes

observable. We mitigate this gap with an iterative framework alternating *self-exploration* and *view graph distillation*, improving Qwen2.5-VL-7B from 2.5% to 47.8% on interactive view planning, surpassing all evaluated frontier models, and inducing spatial priors that transfer to related view-understanding tasks.

Limitations. We study static indoor scenes through a discrete 12-action interface, and validate the framework on two 7–8B backbones; outdoor or dynamic environments, continuous control, and larger model scales are natural next steps. More fundamentally, our trained model still localizes largely by *approaching until the target view becomes observable*, but localizing a target view without ever seeing it, mentally inferring where it lies, remains open. Finally, our framework places little weight on explicit reasoning traces; at our model scale we found it hard to acquire effective explicit reasoning without external expert demonstrations, leaving reasoning-aware training as a further direction.

Acknowledgements

We acknowledge and disclose the use of AI tools in coding and paper writing. We also thank Jiajun Liu, Baiqiao Yin, Jiawei Gu, and Jihan Yang for insightful discussions.

References

- Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2306.13649>.
- John Aloimonos, Isaac Weiss, and Amit Bandyopadhyay. Active vision. *International Journal of Computer Vision*, 1988.
- Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba. Hindsight experience replay. In *Advances in Neural Information Processing Systems*, 2017.
- Anthropic. Claude opus 4.6 system card, 2026. URL <https://www.anthropic.com/claude-opus-4-6-system-card>.
- Daichi Azuma, Taiki Miyanishi, Shuhei Kurita, and Motoaki Kawanabe. Scanqa: 3d question answering for spatial scene understanding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. URL <https://arxiv.org/abs/2112.10482>.
- Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al. Qwen2.5-vl technical report. *arXiv preprint arXiv:2502.13923*, 2025a. URL <https://arxiv.org/abs/2502.13923>.
- Shuai Bai et al. Qwen3-VL technical report. *arXiv preprint arXiv:2511.21631*, 2025b. URL <https://arxiv.org/abs/2511.21631>.
- R. Bajcsy. Active perception. *Proceedings of the IEEE*, 1988.
- Ruzena Bajcsy, Yiannis Aloimonos, and John K. Tsotsos. Revisiting active perception. *Autonomous Robots*, 42(2):177–196, 2018. URL <https://arxiv.org/abs/1603.02729>.
- Neil Burgess. Spatial memory: How egocentric and allocentric combine. *Trends in Cognitive Sciences*, 10(12):551–557, 2006.

- Devendra Singh Chaplot, Dhiraj Gandhi, Saurabh Gupta, Abhinav Gupta, and Ruslan Salakhutdinov. Learning to explore using active neural slam. In *International Conference on Learning Representations (ICLR)*, 2020. URL <https://arxiv.org/abs/2004.05155>.
- Boyuan Chen, Zhuo Xu, Sean Kirmani, Brian Ichter, Danny Driess, Pete Florence, Dorsa Sadigh, Leonidas Guibas, and Fei Xia. Spatialvlm: Endowing vision-language models with spatial reasoning capabilities, 2024. URL <https://arxiv.org/abs/2401.12168>.
- An-Chieh Cheng, Hongxu Yin, Yang Fu, Qiushan Guo, Ruihan Yang, Jan Kautz, Xiaolong Wang, and Sifei Liu. Spatialrgpt: Grounded spatial reasoning in vision-language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL http://papers.nips.cc/paper_files/paper/2024/hash/f38cb4cf9a5eaa92b3cfa481832719c6-Abstract-Conference.html.
- Angela Dai, Angel X. Chang, Manolis Savva, Maciej Halber, Thomas A. Funkhouser, and Matthias Nießner. Scannet: Richly-annotated 3d reconstructions of indoor scenes. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017. URL <https://doi.org/10.1109/CVPR.2017.261>.
- DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025. URL <https://arxiv.org/abs/2501.12948>.
- Ken Deng, Yifu Qiu, Yoni Kasten, Shay B. Cohen, and Yftah Ziser. Lost in space? vision-language models struggle with relative camera pose estimation. *arXiv preprint arXiv:2601.22228*, 2026. URL <https://arxiv.org/abs/2601.22228>.
- Qiyue Gao, Xinyu Pi, Kevin Liu, Junrong Chen, Ruolan Yang, Xinqi Huang, Xinyu Fang, Lu Sun, Gautham Kishore, Bo Ai, Stone Tao, Mengyang Liu, Jiayi Yang, Chao-Jung Lai, Chuanyang Jin, Jiannan Xiang, Benhao Huang, Zeming Chen, David Danks, Hao Su, Tianmin Shu, Ziqiao Ma, Lianhui Qin, and Zhiting Hu. Do vision-language models have internal world models? towards an atomic evaluation. In *Findings of the Association for Computational Linguistics: ACL 2025*, 2025. URL <https://arxiv.org/abs/2506.21876>.
- Google DeepMind. Gemini 3 pro model card, 2025. URL <https://deepmind.google/models/model-cards/gemini-3-pro/>.
- Google DeepMind. Gemini 3.1 pro model card, 2026. URL <https://deepmind.google/models/model-cards/gemini-3-1-pro/>.
- Caglar Gulcehre, Tom Le Paine, Srivatsan Srinivasan, Ksenia Konyushkova, Lotte Weerts, Abhishek Sharma, Aditya Siddhant, Alex Ahern, Miaosen Wang, Chenjie Gu, Wolfgang Macherey, Arnaud Doucet, Orhan Firat, and Nando de Freitas. Reinforced self-training (rest) for language modeling. *arXiv preprint arXiv:2308.08998*, 2023. URL <https://arxiv.org/abs/2308.08998>.
- Wenyi Hong et al. GLM-4.5V and GLM-4.1V-Thinking: Towards versatile multimodal reasoning with scalable reinforcement learning. *arXiv preprint arXiv:2507.01006*, 2025. URL <https://arxiv.org/abs/2507.01006>.
- Yining Hong, Haoyu Zhen, Peihao Chen, Shuhong Zheng, Yilun Du, Zhenfang Chen, and Chuang Gan. 3d-llm: Injecting the 3d world into large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL <https://arxiv.org/abs/2307.12981>.
- Arian Hosseini, Xingdi Yuan, Nikolay Malkin, Aaron Courville, Alessandro Sordoni, and Rishabh Agarwal. V-star: Training verifiers for self-taught reasoners. In *Conference on Language Modeling (COLM)*, 2024. URL <https://arxiv.org/abs/2402.06457>.
- Michael Y. Hu, Benjamin Van Durme, Jacob Andreas, and Harsh Jhamtani. Sample-efficient online learning in lm agents via hindsight trajectory rewriting. *arXiv preprint arXiv:2510.10304*, 2025. URL <https://arxiv.org/abs/2510.10304>.

- Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering, 2023. URL <https://arxiv.org/abs/2308.04079>.
- Kimi Team. Kimi K2.5: Visual agentic intelligence. *arXiv preprint arXiv:2602.02276*, 2026. URL <https://arxiv.org/abs/2602.02276>.
- Xin Lai, Junyi Li, Wei Li, Tao Liu, Tianjian Li, and Hengshuang Zhao. Mini-o3: Scaling up reasoning patterns and interaction turns for visual search. *arXiv preprint arXiv:2509.07969*, 2025. URL <https://arxiv.org/abs/2509.07969>.
- Dingming Li, Hongxing Li, Zixuan Wang, Yuchen Yan, Hang Zhang, Siqu Chen, Guiyang Hou, Shengpei Jiang, Wenqiao Zhang, Yongliang Shen, Weiming Lu, and Yueting Zhuang. Viewspatial-bench: Evaluating multi-perspective spatial localization in vision-language models. *arXiv preprint arXiv:2505.21500*, 2025a. URL <https://arxiv.org/abs/2505.21500>.
- Manling Li, Shiyu Zhao, Qineng Wang, Kangrui Wang, Yu Zhou, Sanjana Srivastava, Cem Gokmen, Tony Lee, Li Erran Li, Ruohan Zhang, Weiyu Liu, Percy Liang, Li Fei-Fei, Jiayuan Mao, and Jiajun Wu. Embodied agent interface: Benchmarking llms for embodied decision making. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2024. URL <https://arxiv.org/abs/2410.07166>.
- Yue Li, Qi Ma, Runyi Yang, Huapeng Li, Mengjiao Ma, Bin Ren, Nikola Popovic, Nicu Sebe, Ender Konukoglu, Theo Gevers, et al. Scenesplat: Gaussian splatting-based scene understanding with vision-language pretraining. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025b. URL <https://arxiv.org/abs/2503.18052>.
- Zhiqiu Lin, Siyuan Cen, Daniel Jiang, Jay Karhade, Hewei Wang, Chancharik Mitra, Tiffany Ling, Yuhua Huang, Sifan Liu, Mingyu Chen, Rushikesh Zawat, Xue Bai, Yilun Du, Chuang Gan, and Deva Ramanan. Towards understanding camera motions in any video. *arXiv preprint arXiv:2504.15376*, 2025. URL <https://arxiv.org/abs/2504.15376>.
- Kevin Lu and Thinking Machines Lab. On-policy distillation. *Thinking Machines Lab: Connectionism*, 2025. URL <https://thinkingmachines.ai/blog/on-policy-distillation>.
- Lu Ma, Hao Liang, Meiyi Qiang, Lexiang Tang, Xiaochen Ma, Zhen Hao Wong, Junbo Niu, Chengyu Shen, Runming He, Yanhao Li, Bin Cui, and Wentao Zhang. Learning what reinforcement learning can’t: Interleaved online fine-tuning for hardest questions. In *International Conference on Learning Representations (ICLR)*, 2026. URL <https://arxiv.org/abs/2506.07527>.
- Wufei Ma, Haoyu Chen, Guofeng Zhang, Yu-Cheng Chou, Jieneng Chen, Celso M de Melo, and Alan Yuille. 3dsrbench: A comprehensive 3d spatial reasoning benchmark, 2025. URL <https://arxiv.org/abs/2412.07825>.
- Xiaojuan Ma, Silong Yong, Zilong Zheng, Qing Li, Yitao Liang, Song-Chun Zhu, and Siyuan Huang. Sqa3d: Situated question answering in 3d scenes. In *International Conference on Learning Representations (ICLR)*, 2023. URL <https://arxiv.org/abs/2210.07474>.
- Arjun Majumdar, Anurag Ajay, Xiaohan Zhang, Pranav Putta, Sriram Yenamandra, Mikael Henaff, Sneha Silwal, Paul McVay, Oleksandr Maksymets, Sergio Arnaud, Karmesh Yadav, Qiyang Li, Ben Newman, Mohit Sharma, Vincent Berge, Shiqi Zhang, Pulkit Agrawal, Yonatan Bisk, Dhruv Batra, Mrinal Kalakrishnan, Franziska Meier, Chris Paxton, Alexander Sax, and Aravind Rajeswaran. Openeqa: Embodied question answering in the era of foundation models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. URL https://openaccess.thecvf.com/content/CVPR2024/html/Majumdar_OpenEqA_Embodied_Question_Answering_in_the_Era_of_Foundation_Models_CVPR_2024_paper.html.
- Yunze Man, De-An Huang, Guilin Liu, Shiwei Sheng, Shilong Liu, Liang-Yan Gui, Jan Kautz, Yu-Xiong Wang, and Zhiding Yu. Argus: Vision-centric reasoning with grounded chain-of-thought. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025. URL <https://arxiv.org/abs/2505.23766>.

- Ida Momennejad, Hosein Hasanbeig, Felipe Vieira Frujeri, Hiteshi Sharma, Nebojsa Jojic, Hamid Palangi, Robert Ness, and Jonathan Larson. Evaluating cognitive maps and planning in large language models with CogEval. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL <https://arxiv.org/abs/2309.15129>.
- Junhyuk Oh, Yijie Guo, Satinder Singh, and Honglak Lee. Self-imitation learning. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2018. URL <https://arxiv.org/abs/1806.05635>.
- John O’Keefe and Lynn Nadel. *The Hippocampus as a Cognitive Map*. Oxford University Press, 1978.
- OpenAI. GPT-5 system card. *arXiv preprint arXiv:2601.03267*, 2026a. URL <https://arxiv.org/abs/2601.03267>.
- OpenAI. GPT-5.4 thinking system card, 2026b. URL <https://deploymentsafety.openai.com/gpt-5-4-thinking>.
- Yulei Qin, Xiaoyu Tan, Zhengbao He, Gang Li, Haojia Lin, Zongyi Li, Zihan Xu, Yuchen Shi, Siqi Cai, Renting Rui, Shaofei Cai, Yuzheng Cai, Xuan Zhang, Sheng Ye, Ke Li, and Xing Sun. Learn the ropes, then trust the wins: Self-imitation with progressive exploration for agentic reinforcement learning. *arXiv preprint arXiv:2509.22601*, 2025. URL <https://arxiv.org/abs/2509.22601>.
- Yuxiao Qu, Tianjun Zhang, Naman Garg, and Aviral Kumar. Recursive introspection: Teaching language model agents how to self-improve. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL <https://arxiv.org/abs/2407.18219>.
- Qwen Team. Qwen3.5: Towards native multimodal agents, 2026. URL <https://qwen.ai/blog?id=qwen3.5>.
- Santhosh Kumar Ramakrishnan, Erik Wijmans, Philipp Krähenbühl, and Vladlen Koltun. Does spatial cognition emerge in frontier models? In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025. URL <https://arxiv.org/abs/2410.06468>.
- Allen Z. Ren, Jaden Clark, Anushri Dixit, Masha Itkina, Anirudha Majumdar, and Dorsa Sadigh. Explore until confident: Efficient exploration for embodied question answering. In *Robotics: Science and Systems (RSS)*, 2024. URL <https://arxiv.org/abs/2403.15941>.
- Stephane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011. URL <https://arxiv.org/abs/1011.0686>.
- Andrei A. Rusu, Sergio Gomez Colmenarejo, Caglar Gulcehre, Guillaume Desjardins, James Kirkpatrick, Razvan Pascanu, Volodymyr Mnih, Koray Kavukcuoglu, and Raia Hadsell. Policy distillation. In *International Conference on Learning Representations (ICLR)*, 2016. URL <https://arxiv.org/abs/1511.06295>.
- Koya Sakamoto, Taiki Miyanishi, Daichi Azuma, Shuhei Kurita, Shu Morikuni, Naoya Chiba, Motoaki Kawanabe, Yusuke Iwasawa, and Yutaka Matsuo. E3vs-bench: A benchmark for viewpoint-dependent active perception in 3d gaussian splatting scenes. *arXiv preprint arXiv:2604.17969*, 2026. URL <https://arxiv.org/abs/2604.17969>.
- Daniel L. Schacter, Donna Rose Addis, and Randy L. Buckner. Remembering the past to imagine the future: The prospective brain. *Nature Reviews Neuroscience*, 8:657–661, 2007.
- Johannes L. Schönberger and Jan-Michael Frahm. Structure-from-motion revisited. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL <https://arxiv.org/abs/2402.03300>.
- Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv:2409.19256*, 2024.
- Roger N. Shepard and Jacqueline Metzler. Mental rotation of three-dimensional objects. *Science*, 171(3972):701–703, 1971.
- Avi Singh, John D. Co-Reyes, Rishabh Agarwal, et al. Beyond human data: Scaling self-training for problem-solving with language models. *Transactions on Machine Learning Research (TMLR)*, 2024. URL <https://arxiv.org/abs/2312.06585>.
- Edward C. Tolman. Cognitive maps in rats and men. *Psychological Review*, 55(4):189–208, 1948.
- Haozhe Wang, Alex Su, Weiming Ren, Fangzhen Lin, and Wenhui Chen. Pixel reasoner: Incentivizing pixel-space reasoning with curiosity-driven reinforcement learning. *arXiv preprint arXiv:2505.15966*, 2025a. URL <https://arxiv.org/abs/2505.15966>.
- Jianyuan Wang, Minghao Chen, Nikita Karaev, Andrea Vedaldi, Christian Rupprecht, and David Novotny. Vggt: Visual geometry grounded transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025b. URL <https://arxiv.org/abs/2503.11651>.
- Kangrui Wang, Pingyue Zhang, Zihan Wang, Yaning Gao, Linjie Li, Qineng Wang, Hanyang Chen, Chi Wan, Yiping Lu, Zhengyuan Yang, Lijuan Wang, Ranjay Krishna, Jiajun Wu, Li Fei-Fei, Yejin Choi, and Manling Li. Vagen: Reinforcing world model reasoning for multi-turn vlm agents. *arXiv preprint arXiv:2510.16907*, 2025c.
- Qineng Wang, Wenlong Huang, Yu Zhou, Hang Yin, Tianwei Bao, Jianwen Lyu, Weiyu Liu, Ruohan Zhang, Jiajun Wu, Li Fei-Fei, and Manling Li. ENACT: Evaluating embodied cognition with world modeling of egocentric interaction. *arXiv preprint arXiv:2511.20937*, 2025d. URL <https://arxiv.org/abs/2511.20937>.
- Qineng Wang, Baiqiao Yin, Pingyue Zhang, Jianshu Zhang, Kangrui Wang, Zihan Wang, Jieyu Zhang, Keshigeyan Chandrasegaran, Han Liu, Ranjay Krishna, Saining Xie, Manling Li, Jiajun Wu, and Li Fei-Fei. Spatial mental modeling from limited views. *arXiv preprint arXiv:2506.21458*, 2025e. URL <https://arxiv.org/abs/2506.21458>.
- Shuzhe Wang, Vincent Leroy, Yohann Cabon, Boris Chidlovskii, and Jerome Revaud. Dust3r: Geometric 3d vision made easy. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. URL <https://arxiv.org/abs/2312.14132>.
- Zihan Wang, Kangrui Wang, Qineng Wang, Pingyue Zhang, Linjie Li, Zhengyuan Yang, Xing Jin, Kefan Yu, Minh Nhat Nguyen, Licheng Liu, Eli Gottlieb, Yiping Lu, Kyunghyun Cho, Jiajun Wu, Li Fei-Fei, Lijuan Wang, Yejin Choi, and Manling Li. Ragen: Understanding self-evolution in llm agents via multi-turn reinforcement learning. *arXiv preprint arXiv:2504.20073*, 2025f.
- Ziyue Wang, Chi Chen, Fuwen Luo, Yurui Dong, Yuanchi Zhang, Yuzhuang Xu, Xiaolong Wang, Peng Li, and Yang Liu. Actiview: Evaluating active perception ability for multimodal large language models, 2025g. URL <https://arxiv.org/abs/2410.04659>.
- Penghao Wu and Saining Xie. V*: Guided visual search as a core mechanism in multimodal LLMs. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. URL <https://arxiv.org/abs/2312.14135>.
- xAI. Grok 4.20 model documentation, 2026. URL <https://docs.x.ai/developers/models/grok-4.20>.

- Jihan Yang, Shusheng Yang, Anjali W. Gupta, Rilyn Han, Li Fei-Fei, and Saining Xie. Thinking in space: How multimodal large language models see, remember, and recall spaces. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025a. URL <https://doi.org/10.1109/CVPR52734.2025.00994>.
- Jihan Yang, Zifan Zhao, Xichen Pan, Shusheng Yang, Junyi Zhang, Bingyi Kang, Hu Xu, and Saining Xie. Cambrian-P: Pose-grounded video understanding, 2026a. URL <https://arxiv.org/abs/2605.22819>.
- Rui Yang, Hanyang Chen, Junyu Zhang, Mark Zhao, Cheng Qian, Kangrui Wang, Qineng Wang, Teja Venkat Koripella, Marziyeh Movahedi, Manling Li, Heng Ji, Huan Zhang, and Tong Zhang. Embodiedbench: Comprehensive benchmarking multi-modal large language models for vision-driven embodied agents. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2025b. URL <https://proceedings.mlr.press/v267/yang25f.html>.
- Shusheng Yang, Jihan Yang, Pinzhi Huang, Ellis Brown, Zihao Yang, Yue Yu, Shengbang Tong, Zihan Zheng, Yifan Xu, Muhan Wang, Daohan Lu, Rob Fergus, Yann LeCun, Li Fei-Fei, and Saining Xie. Cambrian-s: Towards spatial supersensing in video. *arXiv preprint arXiv:2511.04670*, 2025c. URL <https://arxiv.org/abs/2511.04670>.
- Sihan Yang, Runsen Xu, Yiman Xie, Sizhe Yang, Mo Li, Jingli Lin, Chenming Zhu, Xiaochen Chen, Haodong Duan, Xiangyu Yue, Dahua Lin, Tai Wang, and Jiangmiao Pang. Mmsi-bench: A benchmark for multi-image spatial intelligence. In *International Conference on Learning Representations (ICLR)*, 2026b. URL <https://arxiv.org/abs/2505.23764>.
- Yuncong Yang, Han Yang, Jiachen Zhou, Peihao Chen, Hongxin Zhang, Yilun Du, and Chuang Gan. 3d-mem: 3d scene memory for embodied exploration and reasoning. *arXiv preprint arXiv:2411.17735*, 2024. URL <https://arxiv.org/abs/2411.17735>.
- Chun-Hsiao Yeh, Chenyu Wang, Shengbang Tong, Ta-Ying Cheng, Ruoyu Wang, Tianzhe Chu, Yuexiang Zhai, Yubei Chen, Shenghua Gao, and Yi Ma. Seeing from another perspective: Evaluating multi-view understanding in mllms. *arXiv preprint arXiv:2504.15280*, 2025. URL <https://arxiv.org/abs/2504.15280>.
- Naoki Yokoyama, Ram Ramrakhya, Abhishek Das, Dhruv Batra, and Sehoon Ha. HM3D-OVON: A dataset and benchmark for open-vocabulary object goal navigation. *arXiv preprint arXiv:2409.14296*, 2024. URL <https://arxiv.org/abs/2409.14296>.
- Heyang Yu, Yinan Han, Xiangyu Zhang, Baiqiao Yin, Bowen Chang, Xiangyu Han, Xinhao Liu, Jing Zhang, Marco Pavone, Chen Feng, Saining Xie, and Yiming Li. Thinking in 360°: Humanoid visual search in the wild, 2025. URL <https://arxiv.org/abs/2511.20351>.
- Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. Star: Bootstrapping reasoning with reasoning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL <https://arxiv.org/abs/2203.14465>.
- Pingyue Zhang, Zihan Huang, Yue Wang, Jieyu Zhang, Letian Xue, Zihan Wang, Qineng Wang, Keshigeyan Chandrasegaran, Ruohan Zhang, Yejin Choi, Ranjay Krishna, Jiajun Wu, Li Fei-Fei, and Manling Li. Theory of space: Can foundation models construct spatial beliefs through active exploration? In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2026. URL <https://arxiv.org/abs/2602.07055>.
- Tianjun Zhang, Fangchen Liu, Justin Wong, Pieter Abbeel, and Joseph E. Gonzalez. The wisdom of hindsight makes language models better instruction followers, 2023. URL <https://arxiv.org/abs/2302.05206>.
- Xintong Zhang, Zhi Gao, Bofei Zhang, Pengxiang Li, Xiaowen Zhang, Yang Liu, Tao Yuan, Yuwei Wu, Yunde Jia, Song-Chun Zhu, and Qing Li. Adaptive chain-of-focus reasoning via dynamic visual search and zooming for efficient vlms. *arXiv preprint arXiv:2505.15436*, 2025a. URL <https://arxiv.org/abs/2505.15436>.

- Ziang Zhang, Zehan Wang, Guanghao Zhang, Weilong Dai, Yan Xia, Ziang Yan, Minjie Hong, and Zhou Zhao. Dsi-bench: A benchmark for dynamic spatial intelligence, 2025b. URL <https://arxiv.org/abs/2510.18873>.
- Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, Zheyang Luo, Zhangchi Ma, and Yongqiang Ma. Llamafactory: Unified efficient fine-tuning of 100+ language models. *arXiv preprint arXiv:2403.13372*, 2024.
- Yaowei Zheng, Juntong Lu, Shenzhi Wang, Zhangchi Feng, Dongdong Kuang, Yuwen Xiong, and Richong Zhang. Easyr1: An efficient, scalable, multi-modality rl training framework. <https://github.com/hiyouga/EasyR1>, 2025a.
- Ziwei Zheng, Michael Yang, Jack Hong, Chenxiao Zhao, Guohai Xu, Le Yang, Chao Shen, and Xing Yu. Deepeyes: Incentivizing thinking with images via reinforcement learning. *arXiv preprint arXiv:2505.14362*, 2025b. URL <https://arxiv.org/abs/2505.14362>.
- Qian-Yi Zhou, Jaesik Park, and Vladlen Koltun. Open3d: A modern library for 3d data processing. *arXiv preprint arXiv:1801.09847*, 2018. URL <http://arxiv.org/abs/1801.09847>.

Appendix

Table of Contents

A	VIEWSUITE Details	24
A.1	Action Space Details	24
A.2	Data Sampling and Filtering Pipeline	24
A.3	Success Threshold Calibration	26
A.4	View Distance Distribution	26
A.5	Task Examples	27
B	Extended Evaluation Results	29
B.1	Evaluation-Protocol Ablations on IVP	29
B.2	Sample-Level Factor Analysis	30
B.3	Success Factor Definitions	31
C	Iterative Training Implementation Details	33
C.1	Algorithm	33
C.2	RL Hyperparameters	33
C.3	SFT Hyperparameters	33
C.4	View Graph Construction	34
C.5	Task Reformulation Details	35
C.6	Training and Validation Environments	37
D	Extended Analysis	37
D.1	Point Cloud Coverage: Full Model Comparison	37
D.2	Attention Analysis: Methodology and Full Results	37
D.3	Spatial Prior Transfer: Post-Training Details	39

A. VIEWSUITE Details

A.1. Action Space Details

As described in Section 2.2, VIEWSUITE provides 12 camera actions (Figure 1a). Table 7 lists all actions with their geometric definitions. The camera coordinate frame follows the OpenCV convention: +X is screen-right, +Y is screen-down, and +Z points into the scene (forward). The world coordinate frame uses the ScanNet convention with Z-up.

Translation actions. The six translation actions move the camera center along its local axes by $s_t = 0.5\text{ m}$ per step. `move_forward` / `move_backward` translate along the camera’s +Z / -Z axis; `move_left` / `move_right` translate along -X / +X; `move_up` / `move_down` translate along the screen-up / screen-down direction (-Y / +Y under the OpenCV convention).

Rotation actions. The six rotation actions rotate the camera about its center by $s_r = 30^\circ$ per step. `turn_left` / `turn_right` apply yaw (rotation about the camera’s local Y axis); `look_up` / `look_down` apply pitch (rotation about the local X axis); `rotate_ccw` / `rotate_cw` apply roll (rotation about the local Z axis), producing on-screen counter-clockwise / clockwise content rotation.

Discrete snapping. In discrete mode, after each rotation action the camera-to-world rotation matrix is decomposed into intrinsic XYZ Euler angles, each angle is snapped to the nearest multiple of s_r , and the rotation matrix is recomposed. This ensures that camera orientations remain on a regular grid, making action sequences exactly invertible.

Table 7 | Detailed action definitions. All rotations are about the camera center in local coordinates.

Action	Type	Axis	Step size
<code>move_forward</code>	Translation	Camera +Z	0.5 m
<code>move_backward</code>	Translation	Camera -Z	0.5 m
<code>move_left</code>	Translation	Camera -X	0.5 m
<code>move_right</code>	Translation	Camera +X	0.5 m
<code>move_up</code>	Translation	Screen up (-Y)	0.5 m
<code>move_down</code>	Translation	Screen down (+Y)	0.5 m
<code>turn_left</code>	Rotation	Yaw (local Y, -)	30°
<code>turn_right</code>	Rotation	Yaw (local Y, +)	30°
<code>look_up</code>	Rotation	Pitch (local X, +)	30°
<code>look_down</code>	Rotation	Pitch (local X, -)	30°
<code>rotate_ccw</code>	Rotation	Roll (local Z, -)	30°
<code>rotate_cw</code>	Rotation	Roll (local Z, +)	30°

A.2. Data Sampling and Filtering Pipeline

Algorithm 1 gives the pseudocode; Table 8 lists all hyperparameters. Below we describe the four main stages.

Frame sampling. The temporal gap $\delta = f_{\text{tgt}} - f_{\text{init}}$ between initial and target frames is drawn from a mixture over three ranges: $\delta \in [50, 99]$ with weight 0.3, $\delta \in [100, 300]$ with weight 0.5,

Table 8 | Data pipeline hyperparameters.

Hyperparameter	Value
<i>Frame sampling</i>	
$\delta \in [50, 99]$ weight / [100, 300] weight / complement	0.3 / 0.5 / 0.2
Frame span (fraction of video)	[0, 1]
<i>Action planning & filtering</i>	
Axis order	Rot-first
Sequence length bounds $[\ell_{\min}, \ell_{\max}]$	[2, 10]
Max steps per axis (rotation / translation)	12 / 10
<i>Distractor generation</i>	
Num. distractors K	3
Perturb ratio $\lceil r \cdot \ell \rceil$	$r=0.3$
Op probs (replace / remove / insert)	0.6 / 0.2 / 0.2
Same-category replacement prob	0.7
Pixel uniqueness threshold τ	0.02
Max attempts per distractor	20
<i>Limits</i>	
Sampling attempts per pair	20
Timeout per pair	30 s

and the remaining frame indices uniformly with weight 0.2. The heavier weight on larger gaps means most pairs involve substantial viewpoint changes, though some nearby pairs are included as well.

Action planning. Given a sampled pair, we plan an action sequence from the initial to the target camera pose using a greedy, rotation-first strategy (Algorithm 2). The six axes are processed in a fixed order (yaw, pitch, roll, forward, right, up); for each axis we try all step counts up to a maximum and pick the one that most reduces pose error, then commit those steps before moving on. Because the planning is deterministic and single-pass, it consistently produces short sequences. Pairs whose sequence length falls outside [2, 10] are discarded.

Distractor generation. For the multiple-choice P2V and V2P tasks, we create $K=3$ distractors per pair by perturbing the ground-truth action sequence. At $\lceil 0.3 \cdot \ell \rceil$ randomly chosen positions we apply one of three operations (replace with prob. 0.6, remove 0.2, insert 0.2); replacements favor the same motion category with prob. 0.7. Each perturbed sequence is executed and rendered, and we reject any distractor whose mean pixel difference from every existing option is below 0.02.

Scene-level filtering. In addition to the per-pair filters above (viewpoint identity, sequence length), we apply scene-level quality filtering based on the top-down reference view. We first use a VLM to classify each scene’s top-down view as *good* (clear room layout visible from above, floor plan discernible) or *bad* (mostly occluded by ceiling or other geometry, layout not discernible), using 12 few-shot examples (6 good, 6 bad). The automated labels are then manually verified. All view pairs from scenes classified as bad are removed from the dataset. This filtering step removes scenes where the top-down view provides little useful spatial context, which would make the benchmark tasks ill-defined.

Algorithm 1 VIEWSUITE data construction pipeline

Require: Point cloud $\mathcal{P}$; video frames with viewpoints $\{(f_i, P_i)\}_{i=1}^N$; delta distribution $\mathcal{D}$; length bounds $[\ell_{\min}, \ell_{\max}]$; num. distractors K ; pixel threshold τ

- 1: $V_{\text{top}} \leftarrow \text{RENDERTOPDOWN}(\mathcal{P})$
- 2: **for** $n = 1, \dots, N_{\text{pairs}}$ **do**
- 3: Sample $\delta \sim \mathcal{D}$; sample f_{init} ; $f_{\text{tgt}} \leftarrow f_{\text{init}} + \delta$
- 4: $P_{\text{init}}, P_{\text{tgt}} \leftarrow \text{viewpoints at } f_{\text{init}}, f_{\text{tgt}}$
- 5: **if** $P_{\text{init}} \approx P_{\text{tgt}}$ **then skip**
- 6: **end if**
- 7: $\mathbf{a} \leftarrow \text{PLAN ACTIONS}(P_{\text{init}}, P_{\text{tgt}})$ ▷ Alg. 2
- 8: **if** $|\mathbf{a}| \notin [\ell_{\min}, \ell_{\max}]$ **then skip**
- 9: **end if**
- 10: $P_{\text{tgt}}^* \leftarrow \text{EXECUTE}(P_{\text{init}}, \mathbf{a})$ ▷ Snap to discrete grid
- 11: $V_{\text{init}} \leftarrow \text{RENDER}(\mathcal{P}, P_{\text{init}})$; $V_{\text{tgt}} \leftarrow \text{RENDER}(\mathcal{P}, P_{\text{tgt}}^*)$
- 12: $\mathcal{O} \leftarrow \{(\mathbf{a}, V_{\text{tgt}})\}$ ▷ Options: GT first
- 13: **for** $k = 1, \dots, K$ **do** ▷ Generate distractors
- 14: **repeat**
- 15: $\hat{\mathbf{a}} \leftarrow \text{PERTURB}(\mathbf{a})$ ▷ Replace / remove / insert ops
- 16: $\hat{V} \leftarrow \text{RENDER}(\mathcal{P}, \text{EXECUTE}(P_{\text{init}}, \hat{\mathbf{a}}))$
- 17: **until** $\forall (_, V') \in \mathcal{O} : \text{PIXDIFF}(\hat{V}, V') > \tau$
- 18: $\mathcal{O} \leftarrow \mathcal{O} \cup \{(\hat{\mathbf{a}}, \hat{V})\}$
- 19: **end for**
- 20: Emit P2V, V2P, IVP instances from $(V_{\text{init}}, V_{\text{top}}, \mathcal{O})$
- 21: **end for**

A.3. Success Threshold Calibration

We calibrate the threshold multipliers β_t and β_r in the success criterion of Section 2.3 via a small human alignment study. For each rollout we render the agent’s submitted answer viewpoint with the same renderer and intrinsics used at evaluation time, and present it side by side with the ground-truth target view to expert annotators who judge whether the two views depict the same place (*match*) or not. We then sweep (β_t, β_r) over translation thresholds $\{0.25, 0.5, 0.75, 1.0\}$ m and rotation thresholds $\{30^\circ, 60^\circ, 90^\circ\}$, treating the threshold-based success indicator at each setting as a binary classifier of the human label. Table 9 reports precision, recall, F_1 , and accuracy across the resulting 4×3 grid. The combination 0.5 m and 30° , equivalent to $(\beta_t, \beta_r) = (1, 1)$ given the discrete step sizes $s_t = 0.5$ m and $s_r = 30^\circ$, achieves the highest F_1 (0.915) and accuracy (0.920) and is therefore adopted as the default success criterion throughout the paper. Loosening either threshold further keeps recall essentially saturated but degrades precision sharply: with $\beta_t = 2$ (1 m), precision drops to 0.72 at 30° and below 0.6 at 60° , indicating that the human annotators consider many such viewpoints visibly different despite their proximity in pose space.

A.4. View Distance Distribution

Figure 6 shows the empirical distribution of the unified view distance d (Section 2) across the 530 test pairs. The distribution spans roughly 1.4 to 6.8 with mean 3.7, indicating that most pairs require several atomic actions to traverse rather than trivial single-step adjustments. We split the test set at $d = 3$ into a SHORT subset (185 pairs, $d < 3$) and a LONG subset (345 pairs, $d \geq 3$); these subsets are used in the difficulty-stratified analysis of Section 3. The threshold $d = 3$ corresponds to about three atomic-action units of separation between initial and target viewpoints, which empirically produces a clean visual divide between trajectories that can typically be solved with a few correct actions and those that demand sustained planning.

Algorithm 2 Greedy rotation-first action planning

Require: Initial pose P_{init} ; target pose P_{tgt} ; axis order $\mathcal{A} = [\text{yaw}, \text{pitch}, \text{roll}, \text{fwd}, \text{right}, \text{up}]$; max steps per axis k_{max}

Ensure: Action sequence $\mathbf{a}$ such that $\text{EXECUTE}(P_{\text{init}}, \mathbf{a}) \approx P_{\text{tgt}}$

```
1:  $P_{\text{cur}} \leftarrow P_{\text{init}}$ ;  $\mathbf{a} \leftarrow ()$ 
2: for each axis  $(a^+, a^-) \in \mathcal{A}$  do
3:    $e_0 \leftarrow \text{POSEERROR}(P_{\text{cur}}, P_{\text{tgt}})$ 
4:    $k^* \leftarrow \arg \min_{k \in [-k_{\text{max}}, k_{\text{max}}]} \text{POSEERROR}(\text{EXECUTE}(P_{\text{cur}}, k), P_{\text{tgt}}) + 0.01|k|$ 
5:   if  $\text{POSEERROR}(\text{EXECUTE}(P_{\text{cur}}, k^*), P_{\text{tgt}}) < e_0$  then
6:     Append  $|k^*|$  copies of  $(a^+ \text{ if } k^* > 0 \text{ else } a^-)$  to  $\mathbf{a}$ 
7:      $P_{\text{cur}} \leftarrow \text{EXECUTE}(P_{\text{cur}}, k^*)$ 
8:   end if
9: end for
10: return  $\mathbf{a}$ 
```

Table 9 | Success threshold calibration on IVP rollouts. Each row evaluates the threshold-based success indicator at one (position, rotation) threshold pair against the human label. The 0.5 m / 30° setting achieves the best agreement ($F_1=0.915$) and is adopted throughout the paper.

Position thr.	Rotation thr.	Precision	Recall	F1	Accuracy
0.25 m	30°	1.000	0.600	0.750	0.820
0.25 m	60°	0.931	0.600	0.730	0.800
0.25 m	90°	0.931	0.600	0.730	0.800
0.50 m	30°	0.878	0.956	0.915	0.920
0.50 m	60°	0.843	0.956	0.896	0.900
0.50 m	90°	0.843	0.956	0.896	0.900
0.75 m	30°	0.811	0.956	0.878	0.880
0.75 m	60°	0.694	0.956	0.804	0.790
0.75 m	90°	0.683	0.956	0.796	0.780
1.00 m	30°	0.721	0.978	0.830	0.820
1.00 m	60°	0.611	0.978	0.752	0.710
1.00 m	90°	0.587	0.978	0.733	0.680

A.5. Task Examples

We show one example for each of the three tasks, including the system prompt and user prompt given to the model. Images are rendered from ScanNet point clouds. Placeholder [image] tokens mark where images are inserted in the multimodal input.

Path-to-View (P2V). Figure 7 shows a P2V instance. The full prompt is given below.

System: You are a spatial reasoning agent. You are given a question and a set of images. You need to answer the question based on the images. You can think first, which is optional, then answer, respond in this format: `<think>...</think><action>answer(x)</action>` where x is A, B, C, or D.

User: Given the initial view [image] and a top-down reference [image], after you execute the following action sequence (translation step = 0.5 m; rotation step = 30.0 degrees per step): [turn_right, turn_right, turn_right, turn_right, turn_right], which of the following images corresponds to the result? A. [image] B. [image] C. [image] D. [image]

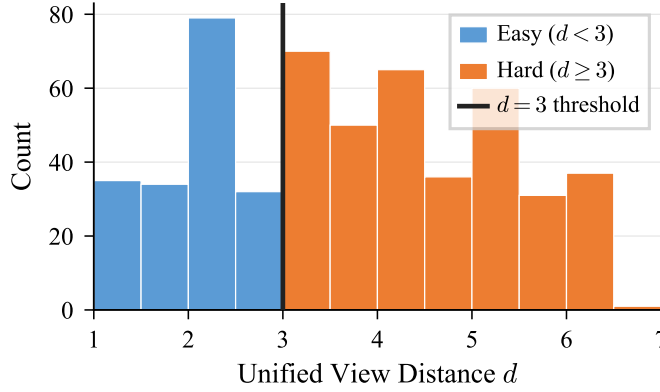


Figure 6 | Distribution of unified view distance d across 530 test pairs. The threshold $d = 3$ separates SHORT (185 pairs) and LONG (345 pairs) subsets.

GPT-5.4 Pro selects option C (incorrect), illustrating that even strong models struggle with large cumulative rotations.

View-to-Path (V2P). Figure 8 shows a V2P instance. The full prompt is given below.

System: You are a spatial reasoning agent. You are given a question and a set of images. You need to answer the question based on the images. You can think first, which is optional, then answer, respond in this format: `<think>...</think><action>answer(x)</action>` where x is A, B, C, or D.

User: Given the initial view [image] and a top-down reference [image], which action sequence will reach the target view [image]? (Action semantics: translation step = 0.5 m; rotation step = 30.0 degrees per step.)

- A. [look_up, move_forward, move_left]
- B. [turn_left $\times 5$, move_left]
- C. [turn_right $\times 2$, move_forward, move_left $\times 5$, move_up]
- D. [turn_left $\times 2$]

GPT-5.4 Pro correctly selects B, reasoning that the target view is behind the initial direction with reversed wall orientation, consistent with a large left rotation plus a lateral shift.

Interactive View Planning (IVP). Figure 9 shows an IVP instance solved by our trained Qwen2.5-VL-7B. The system prompt is given below (abridged; full action list omitted for space).

System: You are solving an interactive view-planning camera pose estimation task.

GOAL: Predict the target view absolute camera pose (camera-to-world, c2w) as a 6-DoF vector: $[tx, ty, tz, rx, ry, rz]$. You may explore the 3D scene using camera-control actions, then submit a final answer. Your predicted pose should be as close as possible to the target pose.

TURN LIMIT: You must complete the task within 10 turns, including the final answer.

OUTPUT FORMAT:

`<think>...</think><action>action_1|action_2|...</action>`. The final response must contain exactly one `answer(tx, ty, tz, rx, ry, rz)`.

User: You're in scene scene0474_00. Please study the target view [image], the initial view [image], and the top-down view [image]. You start from the initial view. Move toward

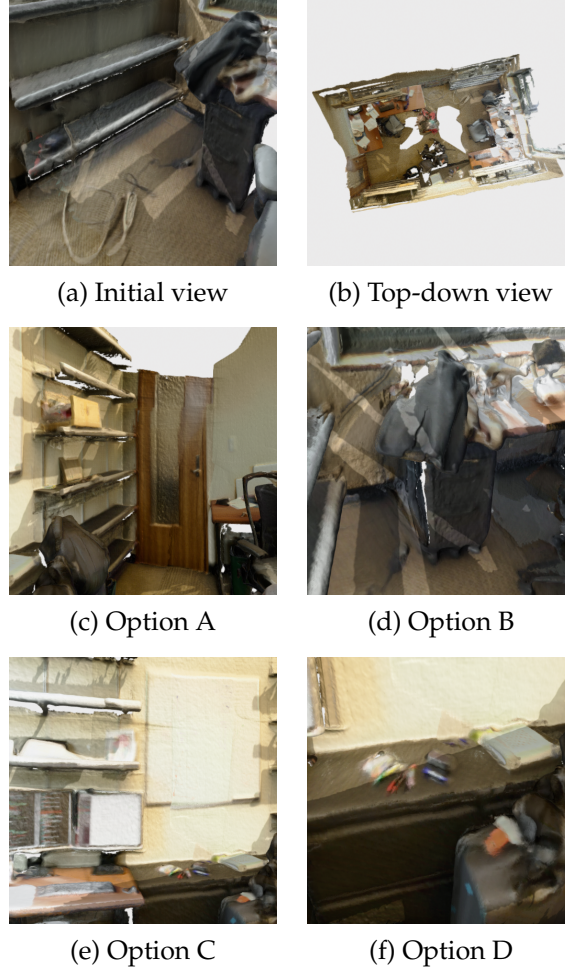


Figure 7 | Path-to-View (P2V) example: given the initial view (a), the top-down view (b), and an action sequence, the model picks the resulting view among options (c)–(f). Action: `[turn_right × 5]`. GPT-5.4 Pro selects C (incorrect).

the target view using actions. Initial view camera 6-DoF: $[tx=4.07, ty=3.28, tz=1.66, rx=-90^\circ, ry=0^\circ, rz=-120^\circ]$. Success thresholds: position error ≤ 0.5 m, rotation error $\leq 30^\circ$.

Over 6 turns, the agent executes: `turn_right` (step 1), `turn_right × 2` (step 2), `turn_right`, `look_down` (step 3), `move_left` (step 4), `move_forward` (step 5), then submits a pose estimate (step 6). The final pose error is 0.061 m position and 0° rotation, well within the success threshold.

B. Extended Evaluation Results

B.1. Evaluation-Protocol Ablations on IVP

The IVP results in Table 2 and Table 5 follow our default evaluation protocol: at each turn the agent’s rotation actions are rounded (*snapped*) to integer multiples of the discrete step size $s_r=30^\circ$, and an episode succeeds only if the agent issues an explicit `submit` action while its pose lies within the unified-distance threshold of the target view. To check that our findings do not hinge on these two protocol details, we re-evaluate our fully-trained models alongside two strong proprietary baselines (Gemini 3.1 Pro and GPT-5.4) under two relaxations:

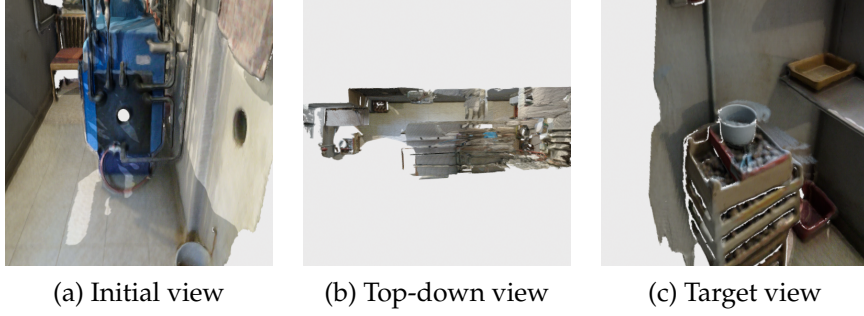


Figure 8 | View-to-Path (V2P) example: given the initial (a), top-down (b), and target (c) views, the model picks which of four action sequences connects the initial view to the target view. GPT-5.4 Pro selects B (correct): `[turn_left × 5, move_left]`.

- **No-Snap.** Per-step rotations are no longer rounded to step-size multiples; the raw rotation magnitudes emitted by the agent are executed as-is. This isolates whether the planning gains depend on the discrete action grid.
- **No-Submit.** The agent does not need to explicitly submit. An episode is counted as successful at the first turn its pose enters the unified-distance threshold of the target, similar to a pure “reach the goal” criterion used in standard navigation benchmarks.

Table 10 compares the default protocol against the two ablations on the Short / Long splits defined in Section 2. The ordering between models is preserved under all three protocols: our trained models continue to outperform Gemini 3.1 Pro and GPT-5.4 by a wide margin (e.g., 19.6 vs. 15.7 on No-Snap, 60.2 vs. 31.5 on No-Submit). Relative to the default protocol, No-Snap *lowers* overall success for every model: without rounding, per-step rotation residuals accumulate over the 10-turn horizon, and the agent drifts off the on-grid pose distribution from which target views are drawn. No-Submit instead *raises* success, since the criterion no longer requires the agent to commit to a final answer and credits any turn at which its pose enters the unified-distance threshold. Across all three protocols the Qwen2.5-VL-7B backbone outperforms Qwen3-VL-8B (47.8 vs. 32.5 on Default, 19.6 vs. 18.5 on No-Snap, 60.2 vs. 48.3 on No-Submit), reinforcing that our framework’s gains transfer to both backbones, with Qwen2.5-VL-7B being the stronger starting point on this benchmark.

Coverage note for GPT-5.4 Pro. GPT-5.4 Pro declines a subset of IVP episodes under its content policy, returning a refusal in place of an action. Of the 530 IVP test instances, 23 are refused in this way and produce no valid rollout; we therefore report GPT-5.4 Pro’s IVP success rate over the remaining 507 valid episodes ($101/507 = 19.9\%$). All other models complete all 530 instances. This affects IVP only: GPT-5.4 Pro completes the full 530 test pairs on both P2V and V2P.

B.2. Sample-Level Factor Analysis

We compute Spearman ρ between 12 sample-level factors and per-model binary success across all three tasks (Figure 10; factor definitions in Appendix B.3). Factors span geometric distance, visual overlap (from pointcloud coverage), and directional geometry.

Across all tasks, distance factors show consistent negative correlations: farther view pairs are harder. For P2V and V2P, orientation agreement is the strongest positive predictor ($\rho \approx +0.19$ – $+0.30$), confirming that same-facing camera pairs are easier to reason about. For IVP, position distance dominates (ρ up to -0.42 for GPT-5.4 Pro), consistent with the position-bottleneck



Figure 9 | Interactive View Planning (IVP) example. Our trained Qwen2.5-VL-7B plans view changes from the initial view (b) to localize the target view (a) in 6 turns. Final pose error: 0.061 m / 0°. Success.

finding in Section 3.2. Visual overlap factors show mild positive correlations for P2V/V2P, indicating that shared visual content helps single-turn prediction. One notable outlier is Grok 4.20 Beta on V2P, which shows near-zero or slightly positive correlations with distance factors, suggesting a qualitatively different (and possibly less spatially grounded) reasoning strategy.

B.3. Success Factor Definitions

We define the 12 sample-level factors used in Figure 10. All factors are computed from the initial and target camera-to-world extrinsics (4×4 matrices) shared across the 530 test view pairs. We write position $\mathbf{t} = C_{[:,3]} \in \mathbb{R}^3$ (the translation column), rotation $R = C_{[:,3:]} \in \mathbb{R}^{3 \times 3}$ (the rotation submatrix), and camera forward direction $\mathbf{f} = -R_{[:,2]} \in \mathbb{R}^3$ (negative z-axis of the camera frame, transformed to world coordinates).

Group A: Geometric Distance.

- `pos_dist` (d_{pos}): $\|\mathbf{t}_{\text{init}} - \mathbf{t}_{\text{target}}\|_2$ (meters). Euclidean distance between camera positions.
- `rot_dist` (d_{rot}): $\arccos(\text{clip}(\frac{\text{tr}(R_{\text{init}}^T R_{\text{target}}) - 1}{2}, -1, 1))$ (degrees). Geodesic angle between orienta-

Table 10 | **The method ranking is protocol-independent.** IVP success rates (%) under the default protocol and two evaluation-protocol ablations. *Default*: per-step rotations are snapped to integer multiples of $s_r=30^\circ$, and success requires an explicit `submit` within the unified-distance threshold; numbers are reproduced from Table 2 and Table 5. *No-Snap*: rotations are not snapped; raw rotation magnitudes are executed as-is. *No-Submit*: no explicit `submit` required; an episode is counted as successful as soon as its pose falls within the unified-distance threshold of the target view. “Ours” denotes our fully-trained models (Section 5.2).

Method	Default			No-Snap			No-Submit		
	Short	Long	All	Short	Long	All	Short	Long	All
Gemini 3.1 Pro	28.6	17.4	21.3	27.0	9.6	15.7	49.7	21.7	31.5
GPT-5.4	33.5	7.5	16.6	27.6	5.2	13.0	57.3	17.4	31.3
Ours (Qwen2.5-VL-7B)	67.2	36.9	47.8	38.9	9.3	19.6	79.5	49.9	60.2
Ours (Qwen3-VL-8B)	56.8	19.4	32.5	41.6	6.1	18.5	80.5	31.0	48.3

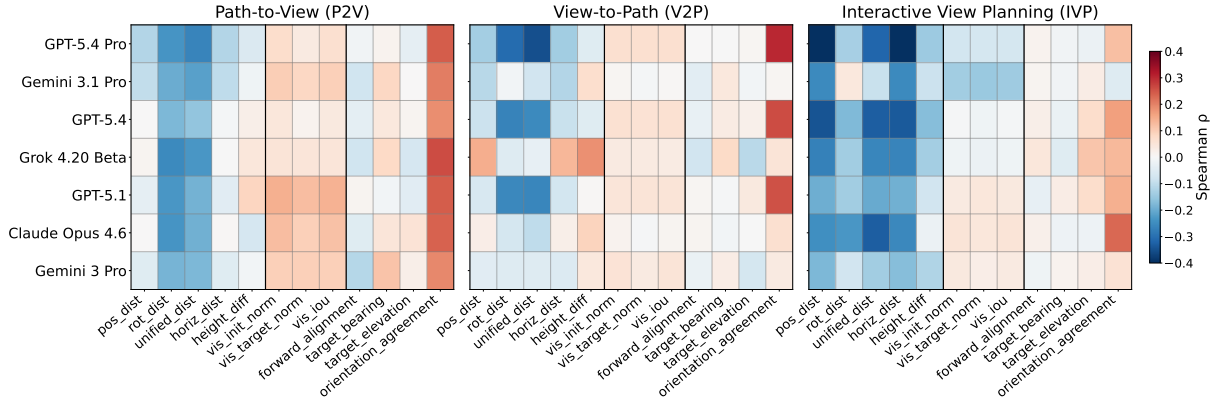


Figure 10 | Spearman ρ between sample-level factors (rows) and per-model success (columns), grouped into geometric distance, visual overlap, and directional geometry. Position distance is the strongest predictor of IVP success; orientation agreement is the strongest for P2V/V2P.

tions.

- **unified_dist**: $\sqrt{(d_{\text{pos}}/s_t)^2 + (d_{\text{rot}}/s_r)^2}$ (steps), where $s_t = 0.5$ m and $s_r = 30^\circ$ are the discrete step sizes in VIEWSUITE. Equivalent to the unified view distance d defined in Section 2.
- **horiz_dist**: $\|\mathbf{t}_{\text{init}}^{\text{xy}} - \mathbf{t}_{\text{target}}^{\text{xy}}\|_2$ (meters). Horizontal distance, ignoring vertical displacement.
- **height_diff**: $|\mathbf{t}_{\text{init}}^z - \mathbf{t}_{\text{target}}^z|$ (meters). Absolute vertical difference.

Group B: Visual Overlap. Computed from GPU-rendered pointcloud coverage: for each viewpoint, we determine which mesh vertices are visible via depth rendering, yielding vertex sets V_{init} and V_{target} .

- **vis_init_norm**: $|V_{\text{init}} \cap V_{\text{target}}| / |V_{\text{init}}|$. Fraction of init-visible vertices also visible from target.
- **vis_target_norm**: $|V_{\text{init}} \cap V_{\text{target}}| / |V_{\text{target}}|$. Fraction of target-visible vertices already visible from init.
- **vis_iou**: $|V_{\text{init}} \cap V_{\text{target}}| / |V_{\text{init}} \cup V_{\text{target}}|$. Intersection-over-union of visible vertex sets.

Group C: Directional Geometry. Let $\hat{\mathbf{d}}$ denote the unit displacement vector from the initial to the target position.

- **forward_alignment:** $\hat{\mathbf{f}}_{\text{init}} \cdot \hat{\mathbf{d}}$. Ranges from +1 (target ahead) to -1 (target behind).
- **target_bearing:** $\arccos(\text{clip}(\text{forward_alignment}, -1, 1))$ (degrees). Angle between init forward direction and displacement to target.
- **target_elevation:** $\text{atan2}(\Delta z, \|\Delta_{xy}\|)$ (degrees). Vertical angle from init to target.
- **orientation_agreement:** $\hat{\mathbf{f}}_{\text{init}} \cdot \hat{\mathbf{f}}_{\text{target}}$. Cosine between camera forward directions. +1 = same facing, -1 = opposite.

C. Iterative Training Implementation Details

C.1. Algorithm

Algorithm 3 summarizes our iterative framework, alternating self-exploration with view graph distillation. Each iteration appends new trajectories to a persistent view graph, samples paths from it, and reformulates them via Eq. 4 into supervised view-planning demonstrations.

Algorithm 3 Self-Exploration with View Graph Distillation

Require: initial policy π_{θ_0} , environments $\mathcal{E}$, iterations K

- 1: $G_0 \leftarrow \emptyset$ ▷ empty view graph
 - 2: **for** $k = 0, 1, \dots, K - 1$ **do**
 - 3: **Self-exploration stage:**
 - 4: Run PPO updates of π_{θ_k} on $\mathcal{E}$ with reward Eq. 3
 - 5: Append trajectories: $G_{k+1} \leftarrow G_k \cup \text{traj}(\pi_{\theta_k})$
 - 6: **View graph distillation stage:**
 - 7: Sample paths $\{P_i\} \subset G_{k+1}$
 - 8: Reformulate: $\mathcal{D}_{k+1} \leftarrow \{\mathcal{R}(P_i)\}$ via Eq. 4
 - 9: Fine-tune via SFT: $\theta_{k+1} \leftarrow \arg \min_{\theta} \mathcal{L}_{\text{SFT}}(\theta; \mathcal{D}_{k+1})$
 - 10: **end for**
 - 11: **return** π_{θ_K}
-

C.2. RL Hyperparameters

Table 11 lists the RL training hyperparameters used across all iterations of our framework and RL baselines. All methods use the same PPO configuration unless otherwise noted.

Iteration-specific overrides. Iterations 0–2 use 60 RL training steps each for rapid bootstrapping. The final iteration (iteration 3) is trained to convergence.

Direct GRPO baseline. The Direct GRPO (filter) baseline uses identical infrastructure but replaces GAE with the GRPO advantage estimator (Shao et al., 2024), sets $n=4$ rollouts per prompt for filtering, and trains for 1,000 steps.

C.3. SFT Hyperparameters

Table 12 lists the SFT hyperparameters used in our framework.

Table 11 | RL training hyperparameters for our framework and baselines.

Hyperparameter	Value
<i>Algorithm</i>	
Advantage estimator	GAE
<i>Actor</i>	
Learning rate	1×10^{-6}
Mini batch size	128
Micro batch size per GPU	2
FSDP param offload	True
FSDP optimizer offload	True
Gradient checkpointing	True
<i>Critic</i>	
Learning rate	1×10^{-5}
Micro batch size per GPU	2
FSDP param offload	True
FSDP optimizer offload	True
Critic warmup steps	0
<i>Rollout</i>	
Engine	SGLang (async)
Max batched tokens	32,768
GPU memory utilization	0.6
Tensor parallel size	1
<i>Data</i>	
Max prompt length	4,000
Max response length	10,000
Train batch size	128
<i>Infrastructure</i>	
GPUs per node	8
Nodes	1

C.4. View Graph Construction

During RL exploration, a background process runs concurrently with training and incrementally merges completed trajectories into the view graph.

Node and edge representation. Each node stores a 6-DoF viewpoint (position and rotation) and its rendered view at 512×512 resolution. Each directed edge stores the sequence of camera actions taken between two viewpoints. Before adding a node, we apply image quality filters: frames with void fraction > 0.7 (indicating the camera is looking outside the point cloud) or pixel standard deviation < 10.0 (indicating a near-uniform, uninformative view) are discarded.

Deduplication. Nodes are deduplicated by viewpoint similarity: a new node is merged with an existing node if their position distance is below 0.25 m *and* rotation distance is below 15° . When two nodes are merged, all edges incident to the new node are redirected to the existing node. Edges are then deduplicated by (source, target, action sequence) identity. This deduplication prevents the graph from growing unboundedly as the same regions of the scene are revisited across episodes.

Table 12 | SFT training hyperparameters for our framework.

Hyperparameter	Value
<i>Training</i>	
Learning rate	1×10^{-5}
Weight decay	0.01
LR scheduler	Cosine
Warmup ratio	0.1
Per-device batch size	2
Gradient accumulation steps	2
Cutoff length	16,384
Precision	BF16
Flash attention	FA2
Distributed strategy	DeepSpeed ZeRO-2
<i>Epochs</i>	
Iterations 0–2	3
Iteration 3 (final)	4
<i>Model selection</i>	
Validation split	20%
Eval strategy	Per epoch
Best model metric	Eval loss

Cross-iteration accumulation. The graph is persisted to disk and accumulated across all self-exploration iterations. Later distillation stages sample from the full exploration history, not just the most recent iteration. This means that spatial knowledge discovered in early iterations (when the policy is weak) remains available for training in later iterations, even if the improved policy explores different regions.

Table 13 shows the graph growth across iterations. The graph grows by an order of magnitude from iteration 0 to iteration 1 as the bootstrapped policy explores more effectively, then grows incrementally in iteration 2. Figure 11 shows how the action distribution shifts across iterations. In iteration 0, `move_forward` dominates (18.0%), reflecting the base policy’s tendency to move straight ahead. By iteration 2, rotations (`turn_left`, `turn_right`) become the most frequent actions (~33% combined), and translations become more balanced across all six directions, indicating that the trained policy has learned more diverse exploration strategies.

Table 13 | View graph growth across iterations (iterations 0–2; the final iteration uses RL only without graph construction). The graph grows by an order of magnitude from iteration 0 to iteration 1, then incrementally.

Iteration	Scenes	Nodes	Edges	Avg Nodes/Scene	Avg Actions/Edge
0	186	4,067	2,875	21.9	1.6
1	193	61,862	62,445	320.5	1.5
2	193	66,492	65,577	344.5	1.7

C.5. Task Reformulation Details

We generate three supervision types from paths sampled from the view graph. Table 14 summarizes the sampling parameters for each task type.

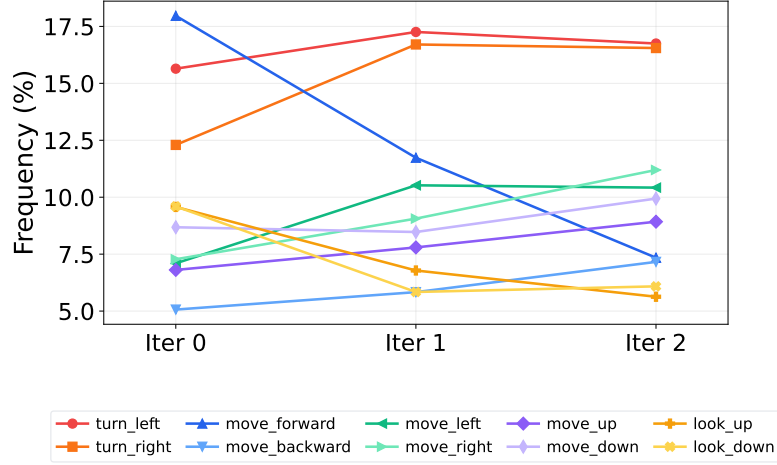


Figure 11 | Action frequency distribution of self-exploration across training iterations. The base policy (iteration 0) favors `move_forward`; later iterations shift toward rotations and more balanced translations.

Table 14 | Task reformulation sampling parameters.

Task Type	Path Length	Samples/Scene	Balanced
Multi-turn view planning	3–5	20	No
View-difference estimation	2–5	15	Yes
View-difference MCQ	2–5	15	Yes

Multi-turn view planning (primary task). For a sampled path of length ℓ ($3 \leq \ell \leq 5$ edges), the end node is designated as the target view and the start node as the initial view. The intermediate nodes provide turn-by-turn observations. The model is trained to predict the correct camera action at each turn, given the current view, the target view, and the planning history. We oversample each path 10 times with different random seeds to increase diversity. This task directly trains the IVP capability.

View-difference estimation. Given two views sampled from nodes at path distance ℓ ($2 \leq \ell \leq 5$), the model predicts the unified view distance between them. This auxiliary task encourages the model to develop a sense of spatial distance between views, complementing the view-planning task. Balanced sampling ensures equal representation across path lengths.

View-difference MCQ. Same setup as view-difference estimation, but presented as a multiple-choice question with four options. This provides an alternative answer format, preventing the model from overfitting to a single task format during SFT.

Additional reformulations (not used in main experiments). The view graph is a task-agnostic representation, and the three tasks above are only the subset we use for training. The same graph naturally admits further reformulations. *Inverse dynamics*: given two views sampled as graph nodes, the model predicts the action sequence labeling the connecting edges. *Forward dynamics*: given an initial view and an action sequence, the model selects the resulting view from several candidate images. We include these to show that distilling structured spatial knowledge

from the graph is not tied to goal-conditioned relabeling; studying their effect on training is left to future work.

C.6. Training and Validation Environments

Training. We use the 3,419 VIEWSUITE training view pairs for RL, each defining one IVP environment (a scene with an initial and a target view). Each episode runs for up to 10 turns at 512×512 image resolution, rendered via Open3d.

Validation. The validation set consists of 378 instances each for the P2V and V2P, and 100 instances for the IVP task.

D. Extended Analysis

D.1. Point Cloud Coverage: Full Model Comparison

Figure 12 extends the coverage analysis from Section 5.3 to all 15 evaluated models. The pattern is consistent: our trained model is the only model that achieves sustained, monotonic growth in target intersection ratio across turns. Frontier proprietary models (GPT-5.4 Pro, Gemini 3.1 Pro, Gemini 3 Pro) show moderate initial increases but plateau or decline after turns 5–7, suggesting they explore broadly without maintaining target-directed trajectories. Open-weight models (Qwen3.5-397B, Qwen2.5-VL-72B, Qwen3-VL-32B) generally remain below the proprietary models in both metrics.

Methodology. For each model, we collect 530 rollout trajectories (585 for GPT-5.4 Pro) on the VIEWSUITE test set. At each turn, we render the agent’s viewpoint against the scene’s 3D point cloud and compute the set of visible vertices using depth-buffered rendering. We track two metrics:

- **Scene coverage ratio:** $|\bigcup_{t=0}^T V_t| / |V_{\text{total}}|$, where V_t is the set of vertices visible at turn t and V_{total} is the full scene point cloud.
- **Target intersection ratio:** $|\bigcup_{t=0}^T V_t \cap V_{\text{target}}| / |V_{\text{target}}|$, where V_{target} is the set of vertices visible from the ground-truth target viewpoint.

All models are evaluated for up to 10 turns (turn 0 is the initial view). Some models produce fewer turns due to early stopping; turns with fewer than 1% of the maximum trajectory count are excluded. Lines show per-turn means; shaded regions indicate ± 1 standard deviation.

Turn distribution and success rate. Figure 13(a) shows the distribution of turns used per rollout. The base Qwen2.5-VL-7B-Instruct and GPT-5.4 Pro terminate most rollouts after a single turn (no planning action taken). Our trained model and Gemini 3.1 Pro concentrate at 10 turns (using all available turns). Figure 13(b,c) show successful rollouts and success rate by turn count. Our trained model achieves high success rates at low turn counts (efficient view planning on easy cases) with decreasing rates at higher turn counts (harder episodes).

D.2. Attention Analysis: Methodology and Full Results

We measure the **image attention fraction** (fraction of response-token attention directed toward image tokens) across all 28 layers on the same 530 trajectories for both models. Figure 5 shows

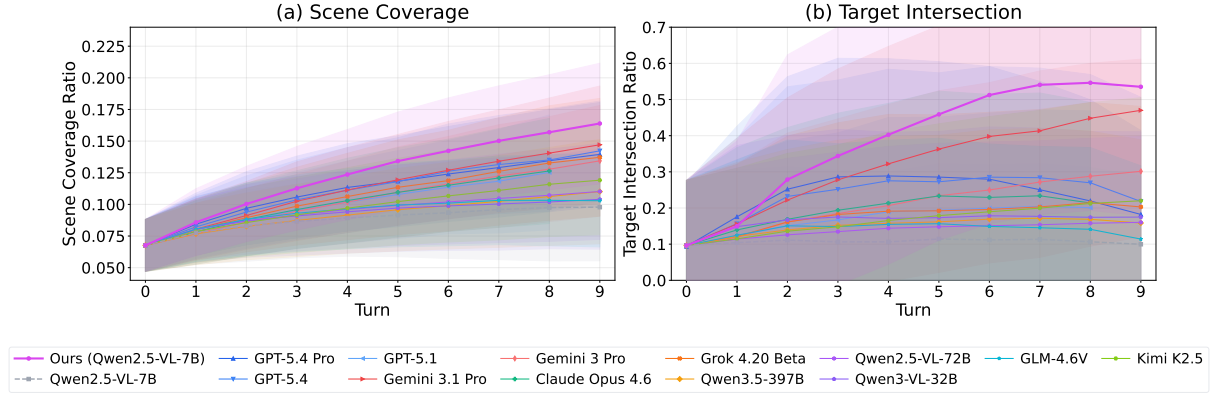


Figure 12 | Point cloud coverage on IVP across all 15 models. (a) Scene coverage ratio (fraction of all scene vertices observed). (b) Target intersection ratio (fraction of target view vertices covered). Only our trained model sustains monotonic growth in target intersection ratio; frontier models plateau or decline after turns 5–7.

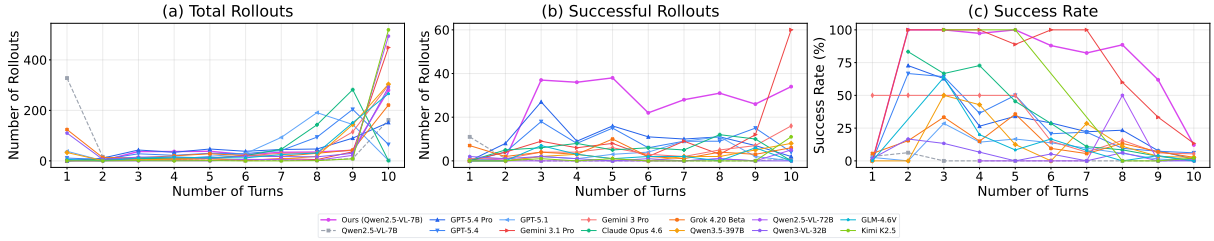


Figure 13 | Turn usage on IVP. (a) Rollouts by turn count: the base model and GPT-5.4 Pro stop after a single turn on most rollouts, while our trained model and Gemini 3.1 Pro use the full 10-turn budget. (b) Successful rollouts and (c) success rate by turn count: our trained model succeeds at high rates on short episodes.

a consistent pattern: the trained model attends more to image tokens than the base model at nearly every layer, and this gap grows with depth (on average 1.4× over layers 0 to 8, 1.9× over layers 9 to 18, and 3.0× over layers 19 to 27). In other words, compared with the base model, the trained model maintains consistently higher attention to the image tokens throughout the layers, suggesting that it learns to use views as evidence for planning rather than relying mainly on textual or prior heuristics.

Setup. We run both our trained model and the base Qwen2.5-VL-7B on the same 530 trajectories. For each trajectory, we perform a single forward pass and extract head-averaged attention from every layer. We identify image token positions via the processor’s token-to-image mapping and compute the **image attention fraction**: for each response token at position q , we measure $\sum_{k \in \mathcal{I}} \alpha_{q,k} / \sum_k \alpha_{q,k}$, where $\mathcal{I}$ is the set of image token indices and $\alpha_{q,k}$ is the attention weight. We report this fraction averaged across response tokens within each turn. Trajectories with fewer than 3 images or no response turns are excluded. Error bars indicate ± 1 standard deviation across trajectories.

Implementation. Qwen2.5-VL-7B has 28 transformer layers. We set *all* layers to “eager” attention so that every layer uses the same computation. Extracting all 28 layers at once exceeds single-GPU memory on long trajectories (up to roughly 6K tokens and 12 images), so we

compute attention in tiles over the query dimension: the computation is exactly eager (a softmax over scaled $QK^\top$ scores), but only one query tile is materialized at a time, and we reduce each tile to the per-token image attention fraction on the fly. All 28 layers are therefore read from a single, fully consistent forward pass per trajectory, and both models use the identical procedure. The full per-layer results are shown in Figure 5 in the main text.

D.3. Spatial Prior Transfer: Post-Training Details

For spatial prior transfer experiments (Section 5.3), we post-train both our trained model and the base Qwen2.5-VL-7B-Instruct using GRPO with identical hyperparameters on each task. We evaluate on three tasks:

- **P2V** and **V2P**: view-action understanding tasks from VIEWSUITE, requiring understanding of how the viewpoint changes under actions. These are trained jointly from the same data splits.
- **MindCube** (Wang et al., 2025e): mental rotation and spatial simulation, requiring the model to track object transformations across viewpoints.

Training hyperparameters. All tasks use GRPO with $n=8$ rollouts per prompt, actor learning rate 1×10^{-6} , and KL penalty disabled ($\lambda_{\text{KL}} = 0$). Training runs for 401 steps on 8 GPUs with FSDP and gradient checkpointing. Table 15 summarizes per-task differences.

Table 15 | Per-task hyperparameters for downstream transfer post-training. All other settings are shared (see text).

	P2V / V2P	MindCube
Train batch size	64	32
Max prompt length	4,000	3,000
Max response length	2,000	4,000

Reward functions. All tasks use a binary reward composed of a format reward and an answer reward. The model must produce a valid response in the format `...<action>...</action>`. For P2V, V2P, and MindCube, the answer reward checks whether the predicted option letter matches the ground truth (case-insensitive first-character match). The reward weights are:

- P2V / V2P: $r = 0.1 \cdot r_{\text{format}} + 0.9 \cdot r_{\text{answer}}$
- MindCube: $r = 0.2 \cdot r_{\text{format}} + 0.8 \cdot r_{\text{answer}}$

Both models are trained for the same number of steps on the same data to ensure a fair comparison of the spatial priors each model brings.